

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KINH TẾ - LUẬT



BÁO CÁO ĐỒ ÁN CUỐI KỲ

Học kỳ I/2025-2026

HỌC PHẦN: HỌC MÁY TRONG PHÂN TÍCH KINH DOANH

**ĐỀ TÀI: TÍCH HỢP MÔ HÌNH HỌC MÁY ĐỂ CÁ NHÂN HOÁ TRẢI NGHIỆM
NGHE NHẠC CỦA NGƯỜI DÙNG**

GVHD: TS. TRẦN DUY THANH

MÃ HP: 251BIM401407

NHÓM SINH VIÊN THỰC HIỆN

STT	Họ và tên	MSSV
1	Nguyễn Hữu Khoa	K234161838
2	Đặng Phương Thảo	K234161854
3	Vũ Lê Phương Thúy	K234161858
4	Trịnh Ngọc Anh Thư	K234161860
5	Lê Tường Vy - nhóm trưởng	K234161875

Thành phố Hồ Chí Minh, tháng 11/2025

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành nhất đến Giảng viên Trần Duy Thanh. Cảm ơn Thầy đã tận tình hướng dẫn và tạo mọi điều kiện thuận lợi để chúng em hoàn thành đồ án môn học này. Nhờ sự hỗ trợ của Thầy, chúng em không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện được nhiều kỹ năng quan trọng như làm việc nhóm, tư duy logic và giải quyết vấn đề.

Trong suốt quá trình học tập, chúng em luôn cảm nhận được sự nhiệt huyết và tâm huyết của Thầy dành cho sinh viên. Những bài giảng đầy giá trị cùng những kinh nghiệm thực tiễn mà Thầy chia sẻ đã trở thành nguồn cảm hứng và hành trang quý báu cho chúng em trên con đường học tập và sự nghiệp sau này. Chúng em đặc biệt trân trọng những góp ý tận tình của Thầy, giúp chúng em từng bước hoàn thiện đồ án này một cách tốt nhất.

Dù đã nỗ lực hết mình, nhóm chúng em không tránh khỏi những thiếu sót trong quá trình thực hiện. Chúng em rất mong nhận được những nhận xét và góp ý quý giá từ Thầy để bài báo cáo có thể hoàn thiện hơn.

Trân trọng,

Moo_d.

MỤC LỤC

LỜI CẢM ƠN	i
DANH MỤC CÁC CHỮ VIẾT TẮT, KÝ HIỆU.....	v
DANH MỤC HÌNH ẢNH	vi
TÓM TẮT DỰ ÁN.....	viii
CHƯƠNG 1: GIỚI THIỆU	1
1.1 Bối cảnh và lý do thực hiện dự án	1
1.2 Vấn đề cần giải quyết	1
1.3 Mục tiêu nghiên cứu	2
1.3.1 Mục tiêu chung	2
1.3.2 Mục tiêu cụ thể	2
1.4 Phạm vi và giới hạn của dự án.....	3
1.4.1 Phạm vi dữ liệu.....	3
1.4.2 Giới hạn của dự án.....	3
1.5 Phương pháp nghiên cứu	3
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	5
2.1 Các khái niệm liên quan	5
2.1.1 Hệ thống gợi ý (Recommender System)	5
2.1.2 Lọc cộng tác (Collaborative Filtering)	5
2.1.3 Lọc theo nội dung (Content-Based Filtering System).....	6
2.1.4 Hệ thống gợi ý Lai (Hybrid Recommender System).....	7
2.1.5 Cách đánh giá và đo lường kết quả	8
2.2 Tổng quan các công trình nghiên cứu	9
CHƯƠNG 3: PHƯƠNG PHÁP THỰC HIỆN.....	11
3.1 Dữ liệu và công cụ sử dụng	11
3.1.1 Dữ liệu	11

3.1.2 Công cụ.....	13
3.2. Mô hình, thuật toán, công nghệ áp dụng	13
3.2.1 Thuật toán:	13
3.2.2 Công nghệ:.....	14
3.3 Xây dựng mô hình học máy	14
3.3.1 Graphic Abstract.....	14
3.3.2 Tổng quan các mô hình nghiên cứu	15
3.3.3 Quy trình triển khai	16
CHƯƠNG 4: KẾT QUẢ VÀ PHÂN TÍCH.....	55
4.1 Kết quả đạt được.....	55
4.1.1 Giao diện đăng ký, đăng nhập tài khoản	55
4.1.2 Giao diện chọn mood.....	56
4.1.3 Giao diện profile.....	57
4.1.4 Giao diện home và các nút chức năng.....	57
4.1.5 Giao diện đĩa nhạc	61
4.1.6 Giao diện lịch sử nghe	62
4.1.7 Giao diện thư viện	63
4.1.8 Giao diện thanh toán.....	65
4.1.9 Giao diện Wrap Up.....	66
4.2. Đánh giá mô hình	68
4.2.1 Mô hình Recommend for today.....	68
4.2.2 Mô hình Recommend artist	69
4.2.3 Mô hình Recommend genres.....	70
4.2.4. Mô Hình Recommend player	72
CHƯƠNG 5: KẾT LUẬN VÀ ĐỀ XUẤT GIẢI PHÁP	76

5.1 Điểm mạnh của dự án.....	76
5.2 Điểm yếu của dự án.....	76
5.3 Hướng phát triển trong tương lai.....	77
5.4 Kết luận.....	78
PHỤ LỤC	79
TÀI LIỆU THAM KHẢO.....	80

DANH MỤC CÁC CHỮ VIẾT TẮT, KÝ HIỆU

STT	Chữ viết tắt, ký hiệu	Nguyên nghĩa
1	CF	Collaborative Filtering
2	CBF	Content-Based Filtering
3	RS	Recommender System
4	TF-IDF	Term Frequency- Invert Document Frequency
5	KNN	K-Nearest Neighbors
6	MAE	Mean Absolute Error
7	RMSE	Root Mean Squared Error
8	MAP@K	Mean Average Precision at K
9	SVD	Singular Value Decomposition

DANH MỤC HÌNH ẢNH

Hình 1: Graphic Abstract.....	14
Hình 2: Ví dụ về LightGBM (Nguồn: FUNiX).....	20
Hình 3: Ma trận user - artist	25
Hình 4: Ví dụ về K-Nearest Neighbours (Nguồn: DataScienceCentral).....	26
Hình 5: Bảng master_features	31
Hình 6: Ví dụ kỹ thuật phân rã ma trận (Nguồn: <i>What is Matrix Factorization?</i> - Ali Zonoozi)	33
Hình 7: Cấu trúc dữ liệu sau khi tách để train model.....	42
Hình 8: Giao diện đăng nhập.....	55
Hình 9: Giao diện đăng ký.....	55
Hình 10: Email người dùng mới.....	56
Hình 11: Giao diện chọn mood	56
Hình 12: Giao diện profile.....	57
Hình 13: Giao diện mô hình Recommend for today	58
Hình 14: Giao diện mô hình Recommend artist.....	58
Hình 15: Giao diện album	59
Hình 16: Danh sách bài hát trong album cụ thể	59
Hình 17: Giao diện mô hình Recommend genres	59
Hình 18: Danh sách bài hát trong genre được gợi ý.....	60
Hình 19: Giao diện chức năng nút sleep timer	61
Hình 20: Giao diện đĩa nhạc.....	62
Hình 21: Giao diện lịch sử nghe nhạc	63
Hình 22: Giao diện thư viện	63
Hình 23: Giao diện bài hát trong Owned Songs.....	64
Hình 24: Giao diện khi tạo playlist.....	64
Hình 25: Giao diện Add song.....	65
Hình 26: Giao diện khi thanh toán bài hát.....	65
Hình 27: Giao diện xác nhận thanh toán	66
Hình 28: Giao diện Wrap Up.....	67
Hình 29: Các trang tiếp theo trong giao diện Wrap Up.....	67

Hình 30: Biểu đồ phân chia mức độ đóng góp của các yếu tố	68
Hình 31: Kết quả đánh giá mô hình LGBM	68
Hình 32: Biểu đồ đánh giá mô hình CF (Top 10).....	70
Hình 33: Biểu đồ phân phối Precision của người dùng.....	71
Hình 34: Biểu đồ phân phối độ đa dạng giữa các bài được gợi ý	72
Hình 35: 3 lớp mô hình Recommend player	73
Hình 36: Kết quả chất lượng mô hình theo từng lớp.....	73
Hình 37: Kết quả chất lượng mô hình	74

TÓM TẮT DỰ ÁN

Đề xuất âm nhạc dựa trên sở thích và cảm xúc của người dùng là một cách để cải thiện quá trình nghe nhạc của họ. Tuy nhiên việc tìm ra điểm tương tự giữa dữ liệu người dùng (ví dụ như lịch sử nghe nhạc, cảm xúc, thói quen, v.v) và âm nhạc là một nhiệm vụ đầy thử thách. Trong dự án này, Moo_d phát triển một số mô hình học máy gợi ý nhạc kết hợp cảm xúc nhằm cá nhân hóa trải nghiệm người dùng trong việc khám phá những bài hát phù hợp. Nhóm sử dụng bộ dữ liệu người dùng giả lập và dữ liệu bài hát từ API Itunes kết hợp hai kỹ thuật: Lọc Cộng Tác (CF) và Lọc Nội Dung (CBF). Tuy nhiên, mô hình hiện tại vẫn còn hạn chế, do tập dữ liệu người dùng giả lập và dữ liệu nhạc thiếu nhiều thuộc tính quan trọng dẫn đến việc chưa khám phá hết tiềm năng của mô hình học máy. Dù vậy, dự án Moo_d hứa hẹn sẽ tăng cường trải nghiệm nghe nhạc của người dùng và mang lại lợi ích kinh doanh đáng kể cho các nền tảng phát nhạc. Qua đó tăng sự gắn kết và kích cầu người dùng mua nhạc số, đáp ứng nhu cầu giải trí ngày càng tăng và thúc đẩy sự phát triển bền vững của dịch vụ.

CHƯƠNG 1: GIỚI THIỆU

1.1 Bối cảnh và lý do thực hiện dự án

Trong lĩnh vực ứng dụng nghe nhạc, thị trường hiện nay vô cùng sôi động với sự xuất hiện của nhiều nền tảng phát nhạc trực tuyến phổ biến như Spotify, Apple Music, YouTube Music, Zing MP3, Nhaccuatui... Những nền tảng này không chỉ đem đến kho nhạc khổng lồ mà còn kết hợp các tính năng thông minh như gợi ý bài hát theo sở thích, tạo playlist cá nhân hóa và hỗ trợ phát nhạc trên nhiều nền tảng.

Bên cạnh đó, nhiều công ty giải trí cũng đầu tư xây dựng trang web nghe nhạc riêng để tiếp cận đến người hâm mộ trực tiếp, điển hình như các ứng dụng của công ty SM Entertainment, BigHit Music (Weverse), hay SoundCloud dành cho các nghệ sĩ độc lập. Điều này cho thấy thị trường ứng dụng nghe nhạc không chỉ là sân chơi của các "ông lớn" mà còn có cơ hội cho những sản phẩm độc lập với tính năng độc đáo, hướng đến những nhóm người dùng cụ thể.

Xuất phát từ bối cảnh trên, cùng với niềm đam mê âm nhạc của từng thành viên đã thúc đẩy nhóm chọn lựa ứng dụng nghe nhạc làm nền tảng chính để kết hợp với mô hình máy học. Ứng dụng Moo_d lần này có thể vẫn còn nhiều thiếu sót, nhưng chắc hẳn đã được cải thiện hơn rất nhiều so với phiên bản trước đó mà nhóm từng xây dựng.

1.2 Vấn đề cần giải quyết

Đứng dưới góc độ người dùng, nhóm nhận thấy rằng việc tìm kiếm bài hát phù hợp với tâm trạng ở các ứng dụng nghe nhạc hiện tại chưa được chú trọng nhiều. Hầu hết các nền tảng tập trung vào đề xuất một chiều - tức là chỉ dựa trên thói quen nghe trong quá khứ - mà không phản chiếu được cảm xúc tức thời. Khi buồn, khi vui, khi cần động lực hay muốn thư giãn, người dùng thường phải tự tìm kiếm hoặc dựa vào những playlist có sẵn từ người dùng khác. Điều này không chỉ ảnh hưởng đến trải nghiệm cá nhân hóa, mà còn làm giảm khả năng liên kết cảm xúc giữa người nghe và âm nhạc.

Từ thực tế đó, nhóm bắt đầu đặt câu hỏi về khả năng xây dựng và phát triển một ứng dụng nghe nhạc có thể hòa hợp cả thói quen nghe và trạng thái cảm xúc của người dùng để nâng cao trải nghiệm người dùng, đó cũng chính là lúc ý tưởng về Moo_d ra

đòi, tên gọi này cũng được lựa chọn vì nó ngụ ý trực tiếp về mục tiêu cốt lõi của ứng dụng. Ứng dụng không chỉ là một nền tảng nghe nhạc thông thường, mà còn kết hợp các mô hình máy học, nhằm đem đến cho người dùng những gợi ý âm nhạc phù hợp với tâm trạng. Nhóm tin rằng đây sẽ là hướng tiếp cận mới mẻ, đồng thời cũng là cơ hội để nhóm thử nghiệm và phát triển một sản phẩm có tính cá nhân hóa cao và khác biệt trên thị trường.

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu chung

Mục tiêu tổng quan của dự án là xây dựng được một hệ thống gợi ý sử dụng các thuật toán học máy thông qua đó đề xuất các sản phẩm âm nhạc lên ứng dụng nghe nhạc Moo_d, giúp đáp ứng và kích cầu người dùng mua nhạc số.

1.3.2 Mục tiêu cụ thể

Các mục tiêu cụ thể về học máy và tính năng mà hệ thống của nhóm cần đạt được là như sau:

Mục tiêu 1: xây dựng mô hình học máy gợi ý Lai (Hybrid System) kết hợp hai thuật toán là Content-Based Filtering (CBF) và Collaborative Filtering (CF) để tận dụng điểm ưu việt của chúng. Mô hình kết hợp này sẽ có khả năng đề xuất được các bài hát có nội dung tương tự với bài mà người dùng đã mua vừa có thể đề xuất ra thêm các nghệ sĩ, bài hát mới mà người dùng đó có tỷ lệ cao là sẽ thích.

Mục tiêu 2: cá nhân hóa các gợi ý của mô hình dựa trên cảm xúc nghe nhạc của người dùng bằng cách tích hợp tích năng Mood Tracking giúp ghi nhận lại mood của người dùng mỗi lần đăng nhập vào ứng dụng, hiện tại nhóm đang có 4 mood (Happy/Sad/Neutral/Intense). Thêm vào đó với các bài hát đã mua người dùng sẽ được gán nhãn mood cho các bài hát này, sau đó hệ thống sẽ gợi ý nhạc có mood tương ứng với mood mà người dùng đã chọn khi đăng nhập ở Mood Tracking.

Mục tiêu 3: từ những phân tích, nghiên cứu trên, triển khai thành công ứng dụng nghe và quản lý mua nhạc số Moo_d, tích hợp trực tiếp mô hình học máy vào ứng dụng để cá nhân hóa trải nghiệm người dùng. Cuối cùng là đo lường hiệu quả bằng các chỉ số máy học.

1.4 Phạm vi và giới hạn của dự án

1.4.1 Phạm vi dữ liệu

Các dữ liệu về thông tin bài hát bao gồm tên bài, nghệ sĩ, link preview, album... đều lấy từ API của Itunes Music. Các dữ liệu còn lại về người dùng (user), lịch sử nghe (history), lịch sử mua (purchase), đánh giá bài hát (user rating), cảm xúc được gán cho bài hát (song moods), lịch sử mood của người dùng (moods tracking history) và dữ liệu tổng hợp hàng tháng của lịch sử nghe và cảm xúc của người dùng sẽ là dữ liệu giả lập.

Về mô hình học máy, dự án sẽ sử dụng 4 mô hình gợi ý khác nhau để tích hợp vào từng giao diện và các chức năng trong ứng dụng.

1.4.2 Giới hạn của dự án

Dự án Moo_d có 2 giới hạn chính:

Bản quyền: Mỗi bài hát chỉ được phát 30 giây preview do vấn đề về bản quyền API của Itunes Music.

Dữ liệu người dùng: Do không có đủ số lượng người dùng thực tế, và không kiếm được một Dataset phù hợp nên nhóm đã quyết định giả lập các dữ liệu liên quan đến người dùng.

1.5 Phương pháp nghiên cứu

Dự án này sử dụng mô hình học máy lai, là mô hình lai kết hợp giữa hai thuật toán CF và CBF. Nhóm sẽ triển khai nghiên cứu theo quy trình như sau:

Giai đoạn 1: Chuẩn bị data, clone meta data của bài hát từ API Itunes và giả lập dữ liệu của người dùng.

Giai đoạn 2: Xây dựng app nghe nhạc Moo_d, các giao diện tương tác, tính năng nghe nhạc, tính năng lưu các giao dịch của người dùng.

Giai đoạn 3: Huấn luyện 4 mô hình học máy đề xuất nhạc số, bao gồm mô hình học máy đề xuất nhạc cho hôm nay (Recommendation for today), gợi ý nhạc theo thể loại âm nhạc (Recommend genres), gợi ý nhạc số theo nghệ sĩ (Recommend artist), và theo bài hát hiện tại (Recommend player).

Giai đoạn 4: Ứng dụng mô hình học máy vào ứng dụng. Tích hợp để mô hình đề xuất và thể hiện được lên mô hình.

Giai đoạn 5: Đánh giá, đo lường kết quả bằng các chỉ số NDCG, MAP, Precision, Recall.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Các khái niệm liên quan

2.1.1 Hệ thống gợi ý (Recommender System)

Hệ thống gợi ý (Recommender System - RS) là một dạng hệ thống lọc thông tin, thường được sử dụng để gợi ý những mục mà người dùng có thể hứng thú với, chẳng hạn sản phẩm, bài hát, bộ phim hay bài báo. RS hoạt động dựa trên nguyên lý quan sát hành vi và sở thích của người dùng trong giai đoạn trước, hoặc dựa trên những người có hành vi tương tự, để rồi đưa ra những gợi ý, dự đoán tương thích với từng cá nhân.

Trong quá trình làm việc, hệ thống sẽ được chuyển hóa thành ma trận tiện ích (utility matrix), hay còn được gọi là ma trận thưa (sparsity matrix). Trong đó, mỗi hàng ngang chịu trách nhiệm biểu diễn người dùng, mỗi cột sẽ đại diện cho từng mục nội dung, và cuối cùng là cái giá trị bên trong ma trận sẽ thể hiện mức độ tương tác, yêu thích của người dùng với các yếu tố trên. Do đó, nhiệm vụ cốt lõi của Recommender System sẽ là dự báo các giá trị còn khuyết và lựa chọn ra những nội dung có tỉ lệ tương thích cao với người dùng để đề xuất. Trong thực tế, RS sẽ được xây dựng theo hai cách chính: gợi ý theo nội dung (Content-Based Filtering), và Lọc cộng tác (Collaborative Filtering).

2.1.2 Lọc cộng tác (Collaborative Filtering)

Hệ thống Lọc cộng tác (Collaborative Filtering - CF) là một trong những kỹ thuật nổi bật nhất, cũng như là một trong các phương pháp căn bản của hệ thống gợi ý. CF hoạt động dựa trên nguyên lý đơn giản nhưng vô cùng hiệu quả: những người có sở thích giống nhau trong quá khứ thì có nhiều khả năng sẽ tiếp tục thích những thứ giống nhau trong tương lai. Vì vậy, thay vì phải phân tích nội dung của từng bài hát hay từng bộ phim, CF theo dõi và phân tích hành vi của người dùng - họ đã nghe gì, xem gì, thích gì và tìm ra những điểm giao nhau giữa những người tương tự.

Trong mô hình CF, dữ liệu thường được biểu hiện dưới dạng ma trận xếp hạng (ratings matrix), trong đó mỗi hàng là một người dùng và mỗi cột là một mục nội dung. Tuy nhiên, ma trận này thường rất thưa vì không phải người dùng nào cũng có thể đánh giá hoặc tương thích với tất cả các nội dung. Nhiệm vụ của CF sẽ là dự đoán những giá

trị còn khuyết dựa trên sự tương đồng. Việc đo độ tương đồng có thể dựa trên các thước đo như Jaccard (khi dữ liệu dạng nhị phân) hoặc Cosine (thường dùng khi dữ liệu đã được chuẩn hóa). Hệ thống Lọc cộng tác thường được thực thi theo hai cách chính:

- Dựa trên bộ nhớ (Memory-based CF): hệ thống trực tiếp so sánh sự giống nhau giữa người dùng với nhau (User-based) hoặc giữa các mục với nhau (Item-based). Từ đó, những gì người tương tự đã thích sẽ được đề xuất cho bạn.
- Dựa trên mô hình (Model-based CF): thay vì so sánh trực tiếp, hệ thống sẽ lọc ra các yếu tố tiềm ẩn (latent factors) - những nét đặc trưng ẩn sau hành vi nghe nhạc và đặc điểm từng bài hát. Cách tiếp cận này thường phân rã ma trận (matrix factorization), trong đó sở thích người dùng và đặc trưng mục được biểu diễn qua tích của hai ma trận U và V , giúp cho dự đoán các giá trị còn thiếu hiệu quả và tinh gọn hơn.

2.1.3 Lọc theo nội dung (Content-Based Filtering System)

Hệ thống Lọc dựa trên nội dung (Content-Based Filtering - CBF) là một cách tiếp cận quen thuộc trong các hệ thống gợi ý, trong đó việc đề xuất được thực hiện dựa trên chính những đặc điểm của các mục và sở thích đã thể hiện của người dùng. Nói ngắn gọn hơn, CBF cố gắng trả lời cho câu hỏi: “Người dùng đã thích những gì trước đây? Và những mục nào có đặc điểm tương tự với chúng?” Từ đó, hệ thống sẽ gợi ý những mục có thuộc tính gần giống với những nội dung mà người dùng đã từng quan tâm.

Để làm được điều này, hệ thống cần xây dựng hồ sơ mục (item profile) - là một tập hợp các đặc điểm mô tả mục đó. Với phim ảnh, các đặc trưng này có thể là diễn viên, đạo diễn, năm sản xuất hoặc thể loại; với bài báo hay tài liệu văn bản, các đặc trưng thường là những từ khóa quan trọng được rút gọn bằng các thống kê như TF-IDF. Hồ sơ của người dùng sau đó được tạo bằng cách tổng hợp đặc trưng của những mục mà người dùng đã thích trước đó. Cuối cùng, mức độ phù hợp giữa người dùng và một mục mới được ước tính thông qua độ tương đồng giữa hai vector đặc trưng - thường phổ biến nhất là khoảng cách cosine.

Tuy nhiên là cách tiếp cận này không phải khi nào cũng hoàn hảo. CBF có xu hướng đề xuất những thứ y chang với những gì người dùng đã biết, dẫn đến việc trải nghiệm đôi khi bị bó hẹp và thiếu sự khám phá. Đồng thời, nếu người dùng mới chưa có nhiều tương tác ban đầu, hệ thống cũng sẽ gặp khó trong việc tạo gợi ý - đây là vấn đề “khởi động nguội” (cold start).

2.1.4 Hệ thống gợi ý Lai (Hybrid Recommender System)

Hệ thống gợi ý Lai (Hybrid Recommender System - Hybrid RS) là một trong ba cách tiếp cận mô hình gợi ý thông qua việc kết hợp nhiều kỹ thuật khác nhau như Lọc cộng tác (CF) và Lọc theo nội dung (CBF) để đưa ra các đề xuất chính xác và phù hợp hơn với từng người dùng. Điểm đặc biệt ở mô hình Hybrid là nó sẽ tận dụng điểm mạnh của từng phương pháp và đồng thời khắc phục những hạn chế còn tồn tại của chúng.

Trong thực tế, CF thường gặp khó khăn với những người dùng hoặc mục mới vì thiếu lịch sử tương tác, còn CBF lại dễ rơi vào trạng thái “chỉ gợi ý những gì quá giống với cái đã thích”, khiến trải nghiệm của người dùng bị lặp lại và hạn chế khám phá mới. Bằng cách kết hợp các phương pháp, Hybrid RS giúp giải quyết cả hai vấn đề: hệ thống có thể đề xuất nội dung ngay cả khi dữ liệu chưa đầy đủ, đồng thời mở rộng phạm vi gợi ý giúp người dùng tiếp cận nhiều nội dung mới mẻ hơn. Một điểm đáng chú ý là Hybrid RS cũng có thể tích hợp thêm yếu tố ngữ cảnh (context), ví dụ như thời gian, tâm trạng, vị trí, hoặc tình huống sử dụng. Điều này sẽ giúp cho việc gợi ý trở nên tự nhiên và “đúng lúc” hơn, phù hợp với đặc điểm trải nghiệm cá nhân hóa mà các hệ thống hiện đại hướng đến.

Ngày nay, với sự phát triển của học sâu (Deep Learning), nhiều mô hình Hybrid hiện đại được triển khai dựa trên việc kết hợp giữa các đặc trưng tường minh (explicit feature interactions) và hệ thống phân cấp đặc trưng (implicit feature hierarchies). Các kiến trúc như Wide & Deep, Neural Factorization Machines (NFM), DeepFM hay Deep & Cross Network đều là những ví dụ tiêu biểu cho hướng tiếp cận này, trong đó hệ thống vừa giữ được khả năng học các mối quan hệ rõ ràng, vừa khai thác được những tầng biểu diễn sâu hơn để làm giàu chất lượng gợi ý.

2.1.5 Cách đánh giá và đo lường kết quả

Dự án này sẽ sử dụng các thước đo xếp hạng (ranking metrics) để đánh giá mô hình. Vì trong hệ thống gợi ý nhạc của Moo_d, do người dùng phải mua nhạc để nghe nên việc đánh giá mô hình cần xem xét trên hai nhiệm vụ riêng biệt:

Gợi ý nhạc đã mua để nghe lại → tập trung vào trải nghiệm và mức độ gắn bó (engagement).

Gợi ý nhạc chưa mua để mua mới → tập trung vào khả năng thúc đẩy hành vi mua (conversion & revenue).

Sau đây là một vài cách đánh giá mô hình học máy mà dự án sẽ sử dụng:

a. Precision

Dùng để đo tỉ lệ bài hát/nghe sĩ mà người dùng sẽ thực sự nghe trong top gợi ý:

$$\text{Công thức: Precision} = \frac{\sum \text{Relevant items}}{\sum \text{All items}}$$

b. Recall

Chỉ số này sẽ giúp ta đánh giá rằng trong số những nghệ sĩ mà user thật sự đã nghe thì mô hình đã gợi ý lại đúng bao nhiêu.

$$\text{Công thức: Recall} = \frac{\# \text{ of our recommendations that are relevant}}{\# \text{ of all the possible relevant items}}$$

c. NDGG

Đánh giá chất lượng xếp hạng (ưu tiên bài liên quan xuất hiện sớm).

$$\text{Công thức: DCG} = \sum_{i=1}^K \frac{\text{rel}_i}{\log_2(i+1)}; \text{ NDCG} = \frac{\text{DCG}}{\text{IDCG}}$$

Trong đó:

rel: là mức độ liên quan (tùy mức label nhưng ví dụ có thể là 0,1)

IDCG: thứ tự mức độ liên quan đúng nhất

d. MAP (Mean Average Precision)

MAP được sử dụng để chỉ ra trung bình độ chính xác trong vị trí của những nghệ sĩ đã được gợi ý đúng.

$$\text{Công thức: } AP = \frac{1}{M} \sum_{k=1}^N P(k) \times \text{rel}_k; \text{ MAP} = \frac{1}{U} \sum_{u=1}^U AP(u)$$

Trong đó:

$P(k)$: Precision tại vị trí k .

rel : 1 nếu nghệ sĩ ở vị trí k được nghe, 0 nếu không.

M : số nghệ sĩ đúng (đã nghe).

U : số user

2.2 Tổng quan các công trình nghiên cứu

Bài báo của (Wang, Xu, Ding, & Tang, 2021) đã phát triển mô hình kết hợp hai thuật toán là Lọc dựa trên nội dung (Content-Based Filtering - CBF) và mạng nơ-ron sâu (Deep Neural Network - DNN) để khai thác đồng thời thuộc tính âm thanh và cảm xúc của người dùng. Kết quả cho thấy phương pháp này sẽ giúp tăng độ chính xác và tính đa dạng của các danh sách đề xuất nhờ việc xem xét song song yếu tố âm thanh và trạng thái cảm xúc của người nghe.

Theo (Madhan, Livin Kumar, Mohammed Abdul Saheen, & Jenifer, 2025), việc sử dụng mô hình học sâu (Deep Learning) để phân tích các đặc trưng âm thanh như nhịp điệu, giai điệu, thể loại và tích hợp nhận diện cảm xúc đã giúp cải thiện đáng kể mức độ hài lòng của người dùng nhờ đề xuất nhạc phù hợp với tâm trạng tức thời. Tương tự, trong nghiên cứu của (Bhutada, và những tác giả khác, 2023) cũng đã thực hiện một khảo sát toàn diện về các hệ thống đề xuất nhạc dựa trên cảm xúc, qua đó nhấn mạnh rằng việc kết hợp dữ liệu cảm xúc và bối cảnh xã hội sẽ làm tăng khả năng giữ chân người dùng và nâng cao tính gắn kết trong trải nghiệm âm nhạc.

(Lu & Wu, 2025) đã thiết kế một hệ thống đề xuất nhạc sử dụng các thuật toán học sâu để phân tích kỹ hành vi và đặc điểm của người dùng. Ứng dụng học sâu và mạng nơ-ron tích chập cho phép mô hình học từ phản hồi cảm xúc ngay trong thời gian thực, giúp hệ thống mô phỏng quá trình ra quyết định của người nghe. Nhờ việc kết hợp

các đặc trưng phân tích được, hệ thống đã đạt hiệu quả vượt trội trong việc nắm bắt sự thay đổi hành vi của người dùng.

Nhìn chung, các công trình nghiên cứu gần đây đã phát triển mạnh mẽ hệ thống gợi ý dựa trên nội dung và hành vi người dùng, hướng đến việc kết hợp cảm xúc, ngữ cảnh và lịch sử nghe nhạc để mang lại trải nghiệm “hiểu người dùng hơn”. Vì thế, ứng dụng Moo_d cũng sẽ kế thừa và phát triển từ những nền tảng nghiên cứu này, tập trung vào việc phân tích cảm xúc người dùng theo thời gian thực, đồng thời phản hồi bằng danh sách bài hát phù hợp với tâm trạng hiện tại. Mục tiêu của Moo_d không dừng lại ở việc gợi ý nhạc đa dạng, mà còn muốn xây dựng một trải nghiệm âm nhạc cá nhân hóa, nơi hệ thống vừa “hiểu cảm xúc” vừa “hỗ trợ điều chỉnh cảm xúc” của người nghe theo hướng tích cực.

CHƯƠNG 3: PHƯƠNG PHÁP THỰC HIỆN

3.1 Dữ liệu và công cụ sử dụng

3.1.1 Dữ liệu

Trong dự án này, nhóm sẽ huấn luyện mô hình học máy dựa trên nguồn dữ liệu gồm ba phần chính là: dữ liệu âm nhạc (music data), dữ liệu người dùng (user data) và dữ liệu tổng hợp (aggregate data). Trong đó, dữ liệu âm nhạc bao gồm các thông tin chi tiết về bài hát, album và nghệ sĩ, sẽ được chúng tôi clone từ iTunes API. Dữ liệu người dùng là phần dữ liệu được tạo ra trong quá trình người dùng tương tác với ứng dụng nhưng do không đủ lượng người dùng thực tế cũng như mong muốn làm đa dạng mô hình nên phần dữ liệu này sẽ được giả lập để có thể phát triển dự án đồng thời cũng đồng bộ với phần dữ liệu âm nhạc. Bên cạnh đó, dữ liệu người dùng còn được tổng hợp theo các chỉ số khác nhằm hỗ trợ các tính năng xếp hạng và phân tích tổng thể.

Dưới đây là một số tập dữ liệu sẽ được sử dụng trong dự án:

	Tập dữ liệu	Thuộc tính	Mô tả
Dữ liệu âm nhạc (music data)	Tracks	trackId, trackName, artistId, artistName, collectionId, collectionName, previewUrl, trackTimeMillis, releaseDate, primaryGenreName, trackNumber, artworkUrl100, popularity	Mỗi bản nhạc (track) được xác định bởi một trackId duy nhất và vì vấn đề bản quyền nên mỗi bài sẽ chỉ phát được 30s preview.
	Collections	collectionId, collectionName, artistId, releaseDate, primaryGenreName, trackCount	Tập hợp các bản nhạc (album hoặc EP) được xác định bởi collectionId.
Dữ liệu người	User	userId, name, email, username, password,	Mỗi người dùng được xác định bằng userId

dùng (user data)		profile_picture, user_type, playlists	và có các thông tin cơ bản khác.
	Purchase	purchaseId, userId, trackId, trackName, artistName, artworkUrl100, category, purchased_at	Ghi lại các giao dịch mua nhạc của người dùng.
	User_history	userId, trackId, trackName, artistName, artworkUrl100, PlayCount, LastPlayedAt	Ghi nhận lịch sử nghe nhạc của người dùng.
	History_month	userId, trackId, trackName, artistName, artworkUrl100, category, played_at	Hệ thống sẽ sử dụng dữ liệu tổng hợp theo tháng (monthly), nguồn dữ liệu này là nền tảng để phân tích sở thích cá nhân và gợi ý nhạc phù hợp cho từng người dùng.
	User_rating	userId, trackId, rating, rateAt	Lưu lại đánh giá của người dùng đối với mỗi track, theo thang đo từ 1-5.
	User_favorite	userId, trackId, trackName, artistName, primaryGenreName, artworkUrl100, added_at	Ghi lại các bài nhạc ưa thích của người dùng.
	Song_moods	trackId, userId, genre, userMood, timestamp	Ghi nhận mood cá nhân mà người dùng gán cho bài hát đó.
	Mood_tracking_history	historyID, userId, moodId, moodName, timestamp	Một tính năng tùy chọn để người dùng

			ghi lại cảm xúc của họ khi nghe nhạc.
Dữ liệu tổng hợp (aggregate data)	Mood_monthly_summary	month, total_entries, dominant_mood, mood_breakdown, userId	Tổng hợp dữ liệu mood hàng tháng của người dùng.
	User_wrapup	userId, month, generated_at, mood_count, dominant_mood, dominant_mood_quote, top_artists, favourite_songs	Báo cáo tổng hợp hàng tháng của người dùng gồm tổng số mood được ghi nhận, top nghệ sĩ và bài hát yêu thích.
	Song_mood_community	trackId, genre, mood_weights, mood_community	Lưu trữ mood chung của mỗi bài hát trên toàn mô hình dựa vào tỷ lệ mood.

3.1.2 Công cụ

Trong quá trình phát triển dự án, đầu tiên nhóm lựa chọn Figma để thiết kế UI/UX, như việc lên ý tưởng xây dựng giao diện người dùng, đảm bảo tính trực quan và trải nghiệm thân thiện trong quá trình sử dụng ứng dụng. Thứ hai, nhóm sử dụng công cụ MongoDB để quản lý dữ liệu vì đặc điểm linh hoạt, phù hợp của nó với các loại dữ liệu phi cấu trúc và thường xuyên thay đổi như lịch sử nghe nhạc và danh sách phát. Bên cạnh đó, việc phát triển và quản lý mã nguồn sẽ được nhóm thực hiện trên PyCharm, kết hợp với GitHub nhằm lưu trữ, quản lý và hỗ trợ nhóm làm việc hiệu quả hơn.

3.2. Mô hình, thuật toán, công nghệ áp dụng

3.2.1 Thuật toán:

Với mong muốn sau khi người dùng nghe bài hát A, hệ thống sẽ gợi ý cho người dùng bài hát B có nội dung tương tự với bài hát A (có thể tương tự về thể loại nhạc, nghệ sĩ), nhóm sẽ dùng thuật toán chính là Content-based Filtering. Ngoài ra, nhóm

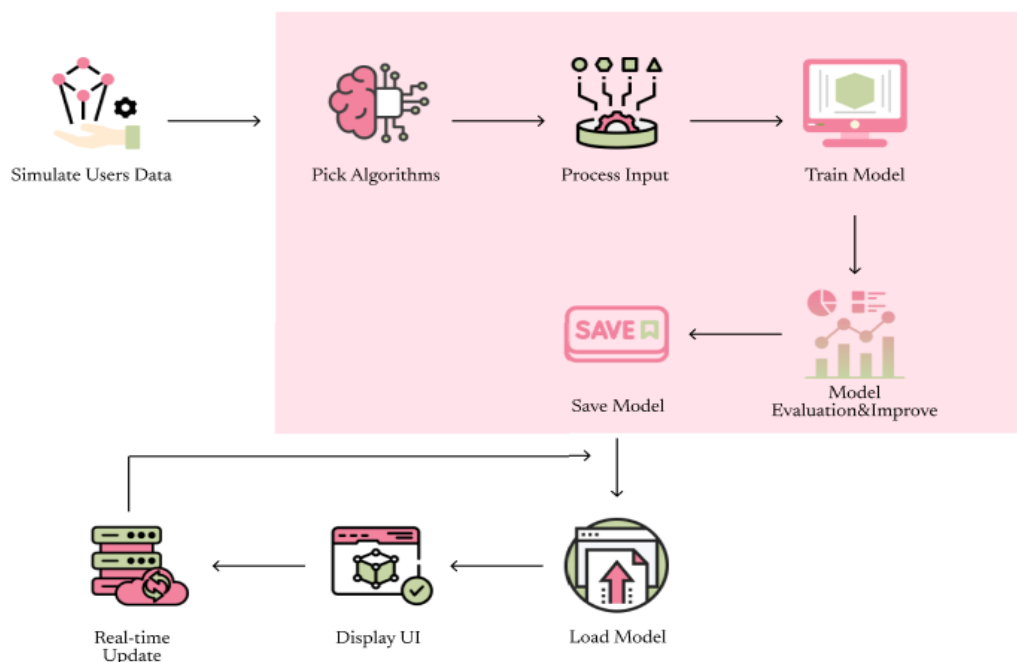
cũng sẽ giả lập thêm nhiều user để dùng thuật toán Collaborative Filtering. Cụ thể với mô hình Recommend for today sẽ chọn nhạc gần giống với mood/thể loại user đã nghe nhiều kết hợp chọn nhạc mà những user tương đồng đang nghe hôm nay (CBF + CF); Mô hình Recommed artist tìm người dùng nghe nghệ sĩ tương tự, rồi gợi ý nghệ sĩ mới (CF); Recommend genres dùng CBF vì mỗi bài đều có track_genre. Cuối cùng là tích hợp mood bằng cách gắn nhãn “happy/sad/chill/neutral” vào feature, rồi lọc nhạc phù hợp.

3.2.2 Công nghệ:

Hiện tại nhóm cần sử dụng Tkinter là thư viện chính để thiết kế giao diện và một số thư viện khác như: PIL (xử lý hình ảnh), vlc (xử lý âm thanh). Song song đó là API Crawling gửi request trực tiếp đến iTunes Search API để lấy dữ liệu (nhạc, artist, genre,...) và lưu dữ liệu vào phần mềm MongoDB.

3.3 Xây dựng mô hình học máy

3.3.1 Graphic Abstract



Hình 1: Graphic Abstract

3.3.2 Tổng quan các mô hình nghiên cứu

3.3.2.1 Mô hình *Recommend for today*

Từ các yếu tố: Thể loại và nghệ sĩ người dùng thích (tức là ở danh sách bài hát yêu thích, xét xem bài hát đó có phần lớn thể loại và nghệ sĩ như thế nào), thời gian phát hành (bài hát nào mới hơn thì giới thiệu trước) và rating hệ thống (ưu tiên bài hát nào có đánh giá từ nhiều người dùng cao hơn), phần “Recommend for today” sẽ đề cử cho người dùng những bài hát phải vừa đáp ứng các yếu tố trên, vừa là những bài mà người dùng chưa mua và trùng với mood người dùng chọn khi vào ứng dụng.

3.3.2.2 Mô hình *Recommend artist*

Mô hình gợi ý nghệ sĩ này được tạo ra là để có thể đem đến cho người dùng trải nghiệm nghe nhạc tuyệt vời hơn từ việc khám phá những nghệ sĩ phù hợp với phong cách cá nhân thông qua việc sử dụng phương pháp Lọc cộng tác lân cận (KKN Collaborative Filtering). Bằng cách phân tích lịch sử nghe, lượt thích, danh sách phát và các tương tác khác của chính người dùng, hệ thống sẽ tìm được “hàng xóm gần nhất” - tức là những người dùng có gu âm nhạc tương đồng với người dùng. Và từ đó sẽ đề xuất những nghệ sĩ chuẩn “gu” thích hợp.

3.3.2.3 Mô hình *Recommend genres*

Mô hình gợi ý theo thể loại được thiết kế để giúp người dùng khám phá thêm các bài chưa mua thuộc thể loại yêu thích của mình. Hệ thống là sự pha trộn giữa hai phương pháp chính: Lọc cộng tác (Collaborative Filtering) - khai thác hành vi nghe nhạc của cộng đồng để phát hiện sở thích tiềm ẩn, và Lọc theo nội dung (Content-Based Filtering) - phân tích đặc trưng của từng bài hát như thể loại, độ phổ biến hay mức độ yêu thích. Việc kết hợp này giúp mô hình vừa nắm bắt được “gu nhạc” của người dùng, vừa đảm bảo khả năng gợi ý đa dạng và phù hợp hơn với từng cá nhân.

3.3.2.4 Mô hình *Recommend player*

Mô hình máy học ở giao diện đĩa xoay được tạo nên dựa trên mô hình sử dụng Content-Based Filtering đa tầng để đề xuất nhạc số phù hợp với sở thích của người dùng nhằm tăng tỷ lệ mua nhạc lên. Vì là giao diện đĩa xoay nhạc để vào được đây user bắt

buộc sẽ phải nghe một bài hát nào đó, nên đầu vào của mô hình sẽ là bài hát đang được phát cùng với cảm xúc mà người dùng đã chọn khi đăng nhập. Để từ đó đề xuất ra 8 bài hát phù hợp với cảm xúc, sở thích nghe nhạc của người dùng. Hệ thống không hybrid các yếu tố trong final score điểm tổng mà điểm score này sẽ được linh hoạt theo từng các trường hợp của user.

Về kiến trúc tổng quan của mô hình, hệ thống bao gồm 3 tầng:

Tầng 1: Pre-Trained Content-Based Filtering đây là tầng mô hình đã được precomputation trước từ trên ứng dụng Google Collab. Ưu điểm là mô hình này đã được xây dựng từ tất cả dữ liệu về người dùng và nhạc có sẵn nên được sử dụng cho các người dùng giả lập của nhóm có thể hiểu ở đây là các người dùng đã sử dụng ứng dụng rồi và đã có dữ liệu. Vì đã được pre-train trước nên tầng này cho ra kết quả tính toán khá là nhanh so với các tầng khác.

Tầng 2: Real-Time Content-Based Filtering đây là tầng dùng trong trường hợp các user cũ đã có dữ liệu rồi nhưng đang nghe các bài hát ít phổ biến hơn không nằm trong top 100 similarity matrix đã tính ở tầng 1, tầng 2 có nhiệm vụ dựa vào behavior đã có từ user đó tìm các similar users tương tự và gợi ý nhạc theo pattern đó.

Tầng 3: Smart Cold Start là tầng dành cho người dùng vừa đăng ký vào ứng dụng Moo_d hoàn toàn chưa có dữ liệu gì về người này, mô hình sẽ dựa vào cảm xúc người đó chọn khi vào Moo_d tìm các bài có điểm cảm xúc đó cao nhất, lấy các bài phổ biến trong genre tương tự với bài đang phát từ đó đề xuất nhạc cho người dùng mới của ứng dụng.

Ba tầng này hoạt động theo cơ chế cơ chế “callback cascade” ưu tiên sử dụng tầng 1 nếu không khả dụng thì chuyển xuống tầng 2 và cuối cùng là tầng 3 đảm bảo cho tất cả các user đều sẽ được đề xuất bài hát phù hợp.

3.3.3 Quy trình triển khai

3.3.3.1 Mô hình *Recommend for today*

Tiền xử lý dữ liệu

```

favorites_with_artist = favorites.merge(tracks[['trackId', 'artistId']], on='trackId',
how='left')
rating_avg = ratings.groupby("trackId")["rating"].mean().reset_index()
rating_avg["rating_score"] = rating_avg["rating"] / 5
# Tạo df chính từ favorites_with_artist
df = favorites_with_artist.merge(song_mood[["trackId", "mood_community"]],
on="trackId", how="left")
df = df.merge(rating_avg[["trackId", "rating_score"]], on="trackId", how="left")
tracks["releaseDate"] = pd.to_datetime(tracks["releaseDate"],
errors="coerce").dt.tz_localize(None)
tracks["recency"] = (pd.Timestamp.now() - tracks["releaseDate"]).dt.days
df = df.merge(tracks[["trackId", "recency"]], on="trackId", how="left")
df["recency_score"] = 1 / (1 + df["recency"].fillna(df["recency"].max()))

```

Đầu vào của mô hình cần có: `rating_score` (điểm đánh giá trung bình của bài hát), `recency_score` (đo độ mới của bài), `artist_similarity` và `genre_similarity` (đo độ tương đồng với nghệ sĩ và thể loại yêu thích) nên bước đầu cần xử lý các dữ liệu này. Trong đó, thêm cột “recency” là số ngày kể từ ngày phát hành cho đến hiện tại, ta tính được “recency_score” = $1/(1 + \text{“recency”})$ tương đương với việc bài càng mới thì “recency_score” càng cao. Còn “rating_score” sẽ lấy điểm đánh giá trung bình của bài hát chia cho 5 để chuẩn hóa về dạng liên tục từ 0 đến 1.

```

# Tạo artist/genre match feature
user_artist_counts = favorites_with_artist.groupby(['userId',
'artistId']).size().reset_index(name='artist_count')
user_genre_counts = favorites_with_artist.groupby(['userId',
'primaryGenreName']).size().reset_index(name='genre_count')
user_total_favs =
favorites_with_artist.groupby('userId').size().reset_index(name='total_favs')
# Merge pre-computed counts vào df
df = df.merge(user_artist_counts, on=['userId', 'artistId'], how='left')
df = df.merge(user_genre_counts, on=['userId', 'primaryGenreName'], how='left')

```

```

df = df.merge(user_total_favs, on='userId', how='left')
# TÍNH similarity bằng vectorization
df['artist_count'] = df['artist_count'].fillna(0)
df['genre_count'] = df['genre_count'].fillna(0)
df['total_favs'] = df['total_favs'].fillna(1) # Tránh chia 0
df["artist_similarity"] = (0.7 * (df['artist_count'] > 0).astype(float) +
                          0.3 * (df['artist_count'] / df['total_favs']))
df["genre_similarity"] = (0.7 * (df['genre_count'] > 0).astype(float) +
                          0.3 * (df['genre_count'] / df['total_favs']))

```

Biến “artist_count” đếm số bài của một nghệ sĩ mà người dùng yêu thích, “total_favs” đếm tổng số bài mà người dùng yêu thích. Từ đó, “artist_similarity” được tính theo công thức $0,7*1 + 0,3*\frac{\text{“artist_count”}}{\text{“total_favs”}}$ nếu có yêu thích nghệ sĩ đó, hoặc theo công thức $0,7*0 + 0,3*\frac{\text{“artist_count”}}{\text{“total_favs”}}$ nếu ở trường hợp ngược lại, với “artist_count”/“total_favs” là tần suất người dùng yêu thích bài của nghệ sĩ. Áp dụng cách tính tương tự, ta sẽ được “genre_similarity”.

```

# Xây dựng mô hình LightGBM với features liên tục
feature_cols = ["recency_score", "rating_score", "artist_similarity",
               "genre_similarity"]
target_col = "cf_score"
X = df[feature_cols]
y = df[target_col]
# Chia train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

```

Bắt đầu vào học máy, như đã trình bày phía trên, những dữ liệu đầu vào của mô hình sẽ bao gồm các yếu tố trong “feature_cols”, còn “target_col” là biến đầu ra.

```

# Tính CF score
cf_counts = favorites_with_artist.groupby("trackId")["userId"].count().reset_index()

```

```
cf_counts.columns = ["trackId", "cf_score"]
cf_counts["cf_score"] = cf_counts["cf_score"] / cf_counts["cf_score"].max()
df = df.merge(cf_counts, on="trackId", how="left")
```

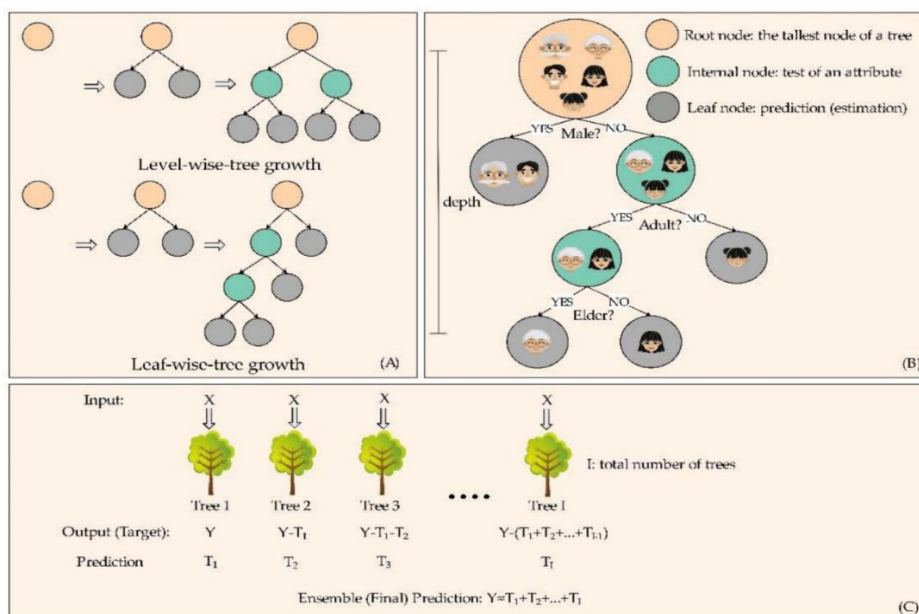
Biến đầu ra “cf_score” được tính bằng cách lấy số người dùng yêu thích bài hát này chia cho số người dùng yêu thích bài hát được yêu thích nhất. Ví dụ, bài A được 20 người thích, bài B có số người thích lớn nhất là 50, thì bài A có “cf_score” = 0,4. Sau khi mô hình học các chỉ số trong feature, nó sẽ dự đoán mức độ phổ biến và yêu thích của bài hát, từ đó xếp hạng điểm dự đoán từ cao đến thấp để lấy top gợi ý cho người dùng.

Dữ liệu sẽ được chia 80% cho training set và 20% test set giúp mô hình được đánh giá khách quan hơn.

```
model = LGBMRegressor(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=6,
    num_leaves=31,
    random_state=42)
model.fit(X_train, y_train)
```

LightGBM (Light Gradient Boosting Machine) là một thuật toán học máy xây dựng nhiều cây nhỏ nối tiếp nhau, cây sau học lỗi của cây trước (tức là “boosting”). Mô hình này sử dụng LightGBM vì nó xử lý được mối quan hệ phi tuyến giữa người dùng và bài hát. Ví dụ, bài A có “artist_similarity” thấp nhưng thuộc thể loại người dùng thường nghe (“genre_similarity” cao) và “rating_score” cao thì vẫn có thể tăng điểm gợi ý để đề xuất cho người dùng. Trong đó, “n_estimators” là số cây, “learning_rate” là tốc độ học, “max_depth” là độ sâu tối đa của mỗi cây (tương đương với việc có tối đa bao nhiêu câu hỏi), “num_leaves” là số lượng lá tối đa trong mỗi cây (tương đương với việc từ số câu hỏi trên, đưa ra tối đa bao nhiêu kết luận).

Sau khi học máy xong, lưu dataframe này lại thành file zip để tái sử dụng mô hình cho những lần sau.



Hình 2: Ví dụ về LightGBM (Nguồn: FUNiX)

Mô hình có sử dụng nghệ sĩ và thể loại mà người dùng thường thích, vậy nên nếu là người dùng mới, chưa có danh sách yêu thích thì mô hình sẽ cho ra kết quả không chính xác - đây chính là vấn đề “Cold Start” điển hình trong các hệ thống gợi ý. Vậy nên, trước khi tích hợp mô hình vào ứng dụng phải phân chia trường hợp cho cẩn thận.

Trường hợp 1: Người dùng mới, chưa có danh sách yêu thích

```
if 'recency' in track_row and pd.notnull(track_row['recency']):
```

```
    days = track_row['recency']
```

```
    if days <= 30:
```

```
        recency_score = 1.0
```

```
    elif days <= 90:
```

```
        recency_score = 0.7
```

```
    elif days <= 180:
```

```
        recency_score = 0.3
```

```
    elif days <= 365:
```

```
        recency_score = 0.1
```

```
    else:
```

```
        recency_score = 0.01
```

```
else:
    recency_score = 0
score += weights['recency'] * recency_score
```

Đối với người dùng mới, “recency_score” sẽ tính theo các trường hợp như trên để ưu tiên các bài hát mới phát hành hơn. Còn “rating_score” sẽ được tính với công thức như cũ.

```
# Top artist score
artist_score = 0.0
if 'artistId' in track_row and pd.notnull(track_row['artistId']):
    artist_popularity = system_top['artist_scores'].get(track_row['artistId'], 0)
    max_artist_pop = max(system_top['artist_scores'].values()) if
system_top['artist_scores'] else 1
    artist_score = artist_popularity / max_artist_pop if max_artist_pop > 0 else 0

score += weights['top_artist'] * artist_score
```

Điểm của nghệ sĩ lần này sẽ dựa vào nghệ sĩ nổi tiếng toàn hệ thống. Đầu tiên, gán “artist_score” = 0, nếu nghệ sĩ có nằm trong top nghệ sĩ phổ biến thì “artist_score” sẽ tính bằng số lượng yêu thích của nghệ sĩ đó chia cho số lượng yêu thích của nghệ sĩ được yêu thích nhất. Với “genre_score”, chúng ta cũng áp dụng cách tính tương tự. Trọng số các thành phần tính điểm cho bài hát cũng được ấn định sẵn với 'recency' là 0.40, 'rating' là 0.30, 'top_artist' là 0.15, 'top_genre' là 0.15.

Vì cần phải gợi ý những bài mà người dùng chưa mua và đúng với mood người dùng chọn lúc vào ứng dụng, cả trường hợp người dùng mới và cũ đều phải có bước lọc để chỉ còn các bài hát đáp ứng yêu cầu trên. Nếu không có bài phù hợp mood mới lấy toàn bộ danh sách để tính điểm gợi ý.

```
artist_diversity_penalty = final_candidates.groupby("artistId").cumcount() *
diversity_weight
```

```
final_candidates["final_score"] = final_candidates["new_user_score"] -  
artist_diversity_penalty  
final_recommendations = final_candidates.nlargest(top_n * 2,  
"final_score").head(top_n)
```

Sau khi tính điểm gợi ý, cần thêm một bước để đa dạng hóa kết quả. Nếu có quá nhiều bài cùng một nghệ sĩ thì giảm nhẹ điểm bằng cách lấy số bài hát được xuất hiện của nghệ sĩ nhân với “diversity_weight” = 0,15 để ra số điểm cần trừ. Tới đây, danh sách gợi ý chính thức cho người dùng mới mới được hình thành.

Trường hợp 2: Người dùng cũ, đã có danh sách yêu thích

Ở trường hợp này, sau khi lọc bài chưa mua và đúng mood cho vào “mood_matched_df” thì dùng mô hình để dự đoán điểm cho từng bài hát dựa trên các feature. Kết quả này cũng phải trừ bớt điểm trùng lặp để đa dạng hóa nghệ sĩ nhưng khác ở chỗ “diversity_weight” = 0,1.

3.3.3.2 Mô hình Recommend artist

```
tracks = pd.read_json(f'{data_path}/tracks.json')  
user_history = pd.read_json(f'{data_path}/user_history.json')  
user_favorite = pd.read_json(f'{data_path}/user_favorite.json')  
user_history["userId"] = user_history["userId"].astype(str)  
user_favorite["userId"] = user_favorite["userId"].astype(str)  
user_history["LastPlayedAt"] = pd.to_datetime(user_history["LastPlayedAt"],  
errors="coerce")  
user_history = user_history.dropna(subset=["LastPlayedAt"])
```

Nhằm cho ra được những nghệ sĩ chất lượng đúng theo gu nghe nhạc của người dùng, đầu vào mô hình sẽ bao gồm các dữ liệu sau: lịch sử nghe nhạc (user_history), những bài hát yêu thích của người dùng (user_favorite), và cuối cùng là thông tin bài nhạc (tracks). Sau đó tiến hành xử lý chuẩn hóa thuộc tính thời gian cuối cùng mà bài hát được nghe (LastPlayedAt) để đảm bảo sự tuyến tính theo mốc thời gian trong quá trình chạy mô hình. Bước kế tiếp, ta chia tỉ lệ huấn luyện học máy là 8:2, tức 80% dữ liệu trước trước đó sẽ được dùng để huấn luyện mô hình và 20% lượt nghe gần nhất sẽ

phục vụ cho mục đích kiểm định. Ở tập train user_favorite thì ta giữ nguyên, không cần chia tỉ lệ.

```
# Giữ lại 20% lượt nghe mới nhất của MỖI USER làm tập test
user_history_sorted = user_history.sort_values(by=['userId', 'LastPlayedAt'])
user_history_sorted['rank'] =
user_history_sorted.groupby('userId')['LastPlayedAt'].rank(method='first',
ascending=True)
user_counts = user_history_sorted.groupby('userId')['rank'].max()
user_history_sorted['cutoff'] = user_history_sorted['userId'].map(user_counts * 0.8)
# Lấy 80% làm train
# Chia history
history_train = user_history_sorted[user_history_sorted['rank'] <=
user_history_sorted['cutoff']]
history_test = user_history_sorted[user_history_sorted['rank'] >
user_history_sorted['cutoff']]
fav_train = user_favorite
```

Từ những dữ liệu trên, hệ thống được chạy trên Google Colab sẽ bắt đầu tính điểm nghe có trọng số dựa trên tần suất nghe nhạc, mức độ yêu thích bài hát của người dùng, và thời gian nghe. Ta sẽ có một số điểm số quan trọng như Recency Weight biểu thị cho trọng số của bài hát sẽ giảm dần kể từ lần nghe gần nhất. Tương tự những bài và nghệ sĩ nào mà được nghe gần đây sẽ có trọng số cao hơn, còn những nghệ sĩ từng nghe mà lâu chưa nghe lại thì trọng số sẽ giảm từ từ theo hàm mũ. Sau đó, hệ số này sẽ được kết hợp với số lượt nghe thực tế để tạo ra trọng số Weighted Play (mức độ quan của người dùng đối với nghệ sĩ nào đó ở thời điểm hiện tại). Bước kế tiếp, hệ thống sẽ rà soát trong dữ liệu bài hát yêu thích của người dùng và bổ sung vào hệ số. Ngoài ra, để tránh cho mô hình chỉ gợi ý những nghệ sĩ ở top đầu thì độ Popularity cũng sẽ được chuẩn hóa, từ đó tăng tính cá nhân và đa dạng hóa cho mô hình. Và ở bước cuối cùng, điểm tổng hợp (final_score) sẽ được tính bằng công thức kết hợp từ WeightedPlay, độ yêu thích của người dùng và mức độ phổ biến của nghệ sĩ nhằm cho ra những gợi ý đảm bảo chất lượng.


```

latest_date_train = history_train["LastPlayedAt"].max()
cutoff_date_train = latest_date_train - pd.DateOffset(months=3)
history_train["RecencyWeight"] = history_train["LastPlayedAt"].apply(
    lambda x: 1 if pd.isnull(x) else np.exp(-((latest_date_train - x).days / 90)))
hist_weighted_train = (
    history_train.groupby(["userId", "artistName"], group_keys=False)
    .apply(lambda df: np.sum(df["PlayCount"] * df["RecencyWeight"]))
    .reset_index(name="WeightedPlay"))
fav_df_train = fav_train.groupby(["userId",
    "artistName"]).size().reset_index(name="favCount")
user_artist_train = pd.merge(hist_weighted_train, fav_df_train,
    on=["userId", "artistName"], how="outer").fillna(0)
latest_date_train = history_train["LastPlayedAt"].max()
cutoff_date_train = latest_date_train - pd.DateOffset(months=3)
history_train["RecencyWeight"] = history_train["LastPlayedAt"].apply(
    lambda x: 1 if pd.isnull(x) else np.exp(-((latest_date_train - x).days / 90)))
hist_weighted_train = (
    history_train.groupby(["userId", "artistName"], group_keys=False)
    .apply(lambda df: np.sum(df["PlayCount"] * df["RecencyWeight"]))
    .reset_index(name="WeightedPlay"))
fav_df_train = fav_train.groupby(["userId",
    "artistName"]).size().reset_index(name="favCount")
user_artist_train = pd.merge(hist_weighted_train, fav_df_train,
    on=["userId", "artistName"], how="outer").fillna(0)
# Thêm Pop (có thể dùng 'tracks' toàn cục)
pop_mean = tracks.groupby("artistName")["popularity"].mean().reset_index()
user_artist_train = pd.merge(user_artist_train, pop_mean, on="artistName",
    how="left").fillna(0)
user_artist_train["popularity_norm"] = user_artist_train["popularity"] /
    user_artist_train["popularity"].max()

```

```
# Tính điểm tổng hợp (chỉ trên data train)
user_artist_train["score"] = user_artist_train["WeightedPlay"] + 2 *
user_artist_train["favCount"]
user_artist_train["final_score"] = (
    user_artist_train["score"] / (1 + user_artist_train["popularity_norm"] ** 1.2))
```

Sau khi đã tính được final_score cho mỗi cặp người dùng và nghệ sĩ, ta sẽ bắt đầu xây dựng ma trận user-artist bằng cách gọi lệnh pivot_train. Ở ma trận này, mỗi hàng sẽ là một artist và mỗi cột sẽ là một người dùng và các giá trị bên trong sẽ đại diện cho mức độ quan tâm của người dùng đối với nghệ sĩ đó.

```
pivot_train = user_artist_train.pivot(index="userId", columns="artistName",
values="final_score").fillna(0)
pivot_sparse_train = csr_matrix(pivot_train.values)
model_train = NearestNeighbors(metric='cosine', algorithm='brute')
model_train.fit(pivot_sparse_train.T)
```

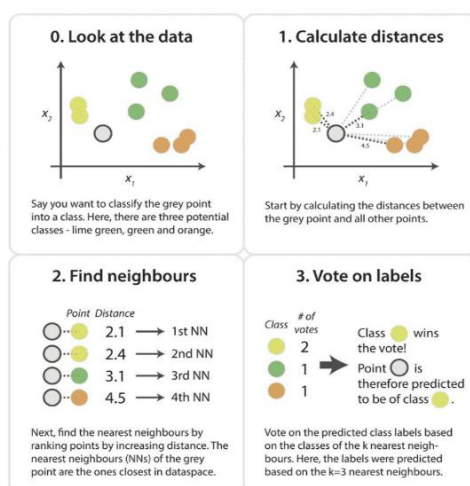
artistName	(S)TRONG	(S)TRONG & Rhymastic	2 Chainz	21 Savage	21 Savage & Brent Faiyaz	21 Savage & Doja Cat	21 Savage & Mariah the Scientist	21 Savage & Metro Boomin	21 Savage & Rich Brian	21 Savage & Summer Walker
userId										
112345	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
112346	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
112347	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
112348	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
112349	0.000	0.000	0.000	12.279	0.000	0.000	0.000	0.000	0.000	0.000
112350	0.000	0.000	0.000	4.955	0.000	0.000	0.800	0.000	0.000	0.000
112351	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
112352	0.000	0.000	0.000	6.098	0.000	0.000	0.000	0.000	0.000	0.000
112353	0.000	0.000	0.000	1.984	0.000	0.000	0.000	0.000	0.000	0.000
112354	0.000	0.000	0.000	0.000	0.000	0.000	0.000	1.357	0.000	0.000

Hình 3: Ma trận user - artist

Để có thể xây dựng mô hình gợi ý nghệ sĩ, ta sẽ sử dụng phương pháp KNN (K-NearestNeighbors) Collaborative Filtering. Đây là một trong những thuật toán tương đối đơn giản và được sử dụng nhiều trong học máy. Thuật toán này sẽ cho kết quả dựa trên sự giống nhau trong hành vi sử dụng của người dùng, nói một cách đơn giản hơn “Ví dụ user A và user B có thói quen nghe nhạc tương tự nhau, thì có khả năng user A sẽ thích những nghệ sĩ giống user B”. Cụ thể, sau khi biểu diễn thói quen nghe nhạc của mỗi user dưới dạng ma trận user - artist như đã trình bày, ta sử dụng độ đo tương đồng cosine (Cosine Similarity) để đo xem hai nghệ sĩ có “gần nhau” trong ma trận sở thích

hay không. Hai nghệ sĩ được xem là tương tự nếu chúng thường cùng xuất hiện trong lịch sử nghe của nhiều người dùng.

Mô hình KNN ở đây đóng vai trò tìm k nghệ sĩ gần nhất so với một nghệ sĩ mà người dùng đã nghe gần đây. Khi người dùng nghe một nghệ sĩ gốc (anchor artist), mô hình sẽ truy xuất ra các nghệ sĩ có véc-tơ phân bố thói quen nghe tương đồng nhất với nghệ sĩ gốc và đề xuất ngược lại cho người dùng.



Hình 4: Ví dụ về K-Nearest Neighbours (Nguồn: DataScienceCentral)

Để mô hình gợi ý nghệ sĩ hiệu quả hơn, ta tiến hành phân chia các trường hợp người dùng để ứng với mỗi tệp người dùng sẽ có một hướng đề xuất phù hợp. Trước đó, ta sẽ phải xác định artist anchor, để lưu ý những nghệ sĩ nào đã được người dùng nghe trong vòng 3 tháng gần nhất. Những nghệ sĩ này chính là gu nhạc hiện tại của người dùng.

Sau đó, hệ thống sẽ bắt đầu tính điểm cho mỗi nghệ sĩ dựa trên điểm `final_score` đã được đề cập trước đó, và `discovery_score` là mức độ khám phá, nhằm ưu tiên những nghệ sĩ không quá nổi.

```
recs = []
for idx, artist in enumerate(top_recent_artists[:3]):
    if artist not in pivot_artist.columns:
        continue
    idx_col = pivot_artist.columns.get_loc(artist)
```

```

dist, ind = model_artist.kneighbors(pivot_sparse.T[idx_col],
n_neighbors=n_neighbors + 1)
weight = 1 - (idx * 0.15)
for d, i in zip(dist[0][1:], ind[0][1:]):
    recs.append({"artist": pivot_artist.columns[i], "sim": 1 - d, "weight": weight})
if not recs:
    return get_popular_artists(top_n)
rec_df = pd.DataFrame(recs).groupby("artist", as_index=False)["weight"].sum()
rec_df.rename(columns={"weight": "final_score"}, inplace=True)
# DIVERSITY WEIGHT
history_all = pd.DataFrame(list(db.user_history.find({}, {"artistName": 1, "_id":
0})))
artist_freq = history_all["artistName"].value_counts(normalize=True)
rec_df["global_freq"] = rec_df["artist"].map(artist_freq).fillna(0)
rec_df["discovery_score"] = rec_df["final_score"] / (1 + 5 * rec_df["global_freq"])

```

Trường hợp 1: Người dùng mới (cold-start), chưa có lịch sử nghe nhạc

```

def get_popular_artists(top_n: int = 10) -> List[Dict[str, Any]]:
    try:
        user_history = pd.DataFrame(
            list(db.user_history.find({}, {"artistName": 1, "_id": 0})))
        if user_history.empty:
            return []
        pop = (
            user_history["artistName"]
            .value_counts()
            .head(top_n)
            .reset_index()
        )
        pop.columns = ["artist", "count"]
        pop["type"] = "Popular"
        return pop[["artist", "type"]].to_dict(orient="records")

```

Đối với những người dùng mới đăng ký tài khoản thì sẽ chưa có lịch sử nghe, thì hệ thống sẽ thống kê 10 nghệ sĩ có lượt nghe nhiều nhất trong `user_history`, sau đó sẽ gợi ý cho người dùng cold start.

```
def recommend_for_user(user_id, top_n=8, n_neighbors=60, months_back=3,
verbose=False):
    try:
        if model_artist is None or pivot_artist is None:
            return get_popular_artists(top_n)
        # LOAD USER HISTORY
        user_hist = pd.DataFrame(list(db.user_history.find(
            {"userId": str(user_id)},
            {"artistName": 1, "PlayCount": 1, "LastPlayedAt": 1, "_id": 0}
        )))
        if user_hist.empty:
            return get_popular_artists(top_n)
```

Trường hợp 2: Người dùng nghe nhiều - heavy (có lượt nghe > 200)

Với tệp người dùng này, hệ thống sẽ cho đề xuất là 5 nghệ sĩ quen thuộc (familiar) với người dùng, thêm với đó sẽ là 3 nghệ sĩ từng thích nhưng lâu rồi chưa thấy nghe lại (re-discover). Hướng tiếp cận này sẽ giúp nhắc lại những nghệ sĩ bị “lãng quên” của người dùng nhưng vẫn giữ được sự chính xác với sở thích của người dùng ở thời điểm hiện tại.

Trường hợp 3: Người dùng nghe trung bình - medium (có lượt nghe từ 50 - 200)

Tuy đã có lịch sử nghe nhưng gu nhạc của tệp người dùng này vẫn còn khác mờ mờ. Thế nên, để tối ưu nhất, hệ thống sẽ kết hợp gợi ý 4 nghệ sĩ quen thuộc (familiar), 2 nghệ sĩ nhắc lại (re-discover) và cuối cùng là sẽ thêm vào 2 nghệ sĩ mới (new diversity). Hai nghệ sĩ mới sẽ được chọn theo chỉ số “discovery_score”, tức sẽ ưu tiên những nghệ sĩ có độ tương đồng cao với người dùng nhưng sẽ ít nổi tiếng hơn, để giúp người dùng tăng sự đa dạng của nghệ ý.

Trường hợp 4: Người dùng nghe ít - light (có < 50 lượt nghe)

Đây là nhóm người dùng có rất ít dữ liệu, thế nên mô hình CF chưa thể hoạt động hiệu quả giữa nhóm người này, vì vậy rất khó đưa ra được dự đoán gu nhạc. Hệ thống sẽ ưu tiên phương pháp fallback, tức ta sẽ ưu tiên lấy 3 nghệ sĩ quen thuộc (nếu đủ), sau đó gợi ý thêm những nghệ sĩ phổ biến nhất theo hệ thống (popular). Việc này sẽ giúp đưa ra những gợi ý an toàn, giúp người dùng quen với app nhanh hơn.

```
if mode == "heavy":
    selected = pd.concat([familiar.head(5), rediscover.head(3)])
elif mode == "medium":
    selected = pd.concat([familiar.head(4), rediscover.head(2), new_div.head(2)])
else: # light user
    pop = pd.DataFrame(get_popular_artists(50))
    # 3 familiar + 5 popular/diversity
    f = familiar.head(3)
    # 5 phổ biến
    p = pop[~pop["artist"].isin(f["artist"])].head(5)
    selected = pd.concat([f, p])
    # NEW: EP FILL ĐỦ 8 NGƯỜI
    if len(selected) < 8:
        need = 8 - len(selected)
        # chọn từ new diversity
        fill = new_div[~new_div["artist"].isin(selected["artist"])].head(need)
        selected = pd.concat([selected, fill])
    # nếu vẫn thiếu → fallback toàn hệ thống
    if len(selected) < 8:
        need = 8 - len(selected)
        extra_pop = pop[~pop["artist"].isin(selected["artist"])].head(need)
        selected = pd.concat([selected, extra_pop])

# Drop trùng và giữ đúng số lượng
selected = selected.drop_duplicates("artist").head(top_n)
```

3.3.3.3 Mô hình *Recommend genres*

a. Chuẩn bị dữ liệu

Cũng tương tự như các mô hình trước, vì được huấn luyện trên Google Colab nên bước đầu tiên sẽ kết nối tới Drive nơi chứa các file dữ liệu giả lập và giải nén ra một thư mục riêng bằng cách sử dụng thư viện zipfile. Các file dữ liệu cần dùng cho mô hình sau đó được đọc vào bằng pandas `pd.read_json` để chuyển thành DataFrame và chuẩn hóa kiểu dữ liệu, bao gồm các bảng sau:

- `tracks`: lấy những thông tin của bài hát (`trackId`, `artistId`, `collectionName`, `trackTimeMillis`, `primaryGenreName`...)
- `user_history`: lịch sử nghe nhạc (`userId`, `trackId`, `playCount`, `LastPlayedAt`,...)
- `user_rating` / `user_favorites` / `purchase`: các bảng phụ trợ về lượt mua, lượt thích và đánh giá bài hát

b. Tạo bảng tổng hợp thuộc tính của bài hát - master features

Thông qua hàm `create_master_features_df`, các bảng dữ liệu đầu vào được tổng hợp và chuẩn hóa để tạo ra bảng `master_features`. Cụ thể các bước tạo ra như sau:

Bước 1. Tổng hợp các thuộc tính cơ bản của bài hát từ bảng `tracks`, bao gồm `'trackId'`, `'trackName'`, `'artistId'`, `'artistName'`, `'collectionName'`, `'primaryGenreName'`, `'releaseDate'`, `'trackTimeMillis'`

Bước 2. Feature 1: thuộc tính `'popularity_score'` được tính bằng tổng số lượt nghe và tổng số người dùng độc lập gộp từ bảng `user_history`, cho thấy độ phổ biến của bài hát.

Bước 3. Feature 2: thuộc tính `'recency_score'` cho biết độ mới của bài hát $master_df['recency_score'] = 1 - (master_df['days_since_release'] / max_days)$, bài hát càng ra mắt gần đây thì điểm càng cao.

Bước 4. Feature 3: thuộc tính `'favorite_count'` đếm lượt yêu thích của bài hát từ bảng `user_favorite`

Bước 5. Feature 4: thuộc tính `'avg_rating'` là điểm đánh giá trung bình của bài hát.

Bước 6. Feature 5: thuộc tính ‘rating_count’ đếm lượt đánh giá.

Bước 7. Feature 6: thuộc tính binary ‘is_purchased’ để xét xem bài đã từng được mua hay chưa.

Bảng master_features trả về sẽ có cấu trúc như hình và sẽ được lưu về dưới dạng file .pkl để sử dụng trong các giải thuật về sau.

```
RangeIndex: 17950 entries, 0 to 17949
Data columns (total 17 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   trackId                17950 non-null  object
1   trackName              17950 non-null  object
2   artistId               17950 non-null  object
3   artistName             17950 non-null  object
4   collectionName         17950 non-null  object
5   primaryGenreName       17950 non-null  object
6   releaseDate            17937 non-null  datetime64[ns, UTC]
7   trackTimeMillis        17950 non-null  float64
8   total_plays            17950 non-null  float64
9   unique_listeners       17950 non-null  float64
10  popularity_score       17950 non-null  float64
11  days_since_release     17950 non-null  float64
12  recency_score          17950 non-null  float64
13  favorite_count         17950 non-null  float64
14  avg_rating             17950 non-null  float64
15  rating_count           17950 non-null  float64
16  is_purchased           17950 non-null  int64
dtypes: datetime64[ns, UTC](1), float64(9), int64(1), object(6)
memory usage: 2.3+ MB
```

Hình 5: Bảng master_features

c. Tạo ma trận tương tác user - item matrix

Sau khi đã có master_feature, bước tiếp theo trong quy trình là trích xuất và xử lý lịch sử nghe để làm dữ liệu đầu vào cho mô hình. Hàm prepare_simple_history_data sẽ ghi nhận các tương tác của người dùng từ bảng user_history rồi bắt đầu các bước tính toán:

- Tính độ mạnh của tương tác (strength): chỉ số này phản ánh mức độ hứng thú của người dùng đối với một bài hát, được tính bằng hai thành phần chính: độ mới và tần suất của tương tác đó.

```
history_clean['timestamp'] = pd.to_datetime(history_clean['LastPlayedAt'],
errors='coerce')
history_clean = history_clean.dropna(subset=['timestamp'])
current_time = history_clean['timestamp'].max()
```



```

history_clean['days_ago'] = (current_time - history_clean['timestamp']).dt.days
history_clean['days_ago'] = history_clean['days_ago'].clip(lower=0)
max_days = history_clean['days_ago'].max() or 1
history_clean['recency_strength'] = 1 - (history_clean['days_ago'] / max_days)
max_play = history_clean['PlayCount'].max() or 1
history_clean['play_strength'] = history_clean['PlayCount'] / max_play
history_clean['strength'] = (
    (1 - recency_weight) * history_clean['play_strength'] +
    recency_weight * history_clean['recency_strength'])

```

- Chia tập dữ liệu huấn luyện - kiểm thử (train - test set): sử dụng hàm `sklearn.model_selection.train_test_split` theo tỉ lệ `test_size = 0.2`, chia theo từng user để đảm bảo user có interaction ở cả hai tập dữ liệu. Kết quả sau khi chia gồm 326843 interactions cho tập huấn luyện và 82200 interactions cho tập kiểm thử.
- Tạo ma trận tương tác người dùng - bài hát (user – item interaction matrix): sử dụng `pivot_table` trên tập dữ liệu huấn luyện. Trong ma trận này, người dùng sẽ là hàng (index), bài hát là các cột (columns), giá trị tại mỗi ô chính là chỉ số strength đã được tính toán ở bước đầu. Các ô chưa có tương tác được fill giá trị = 0.

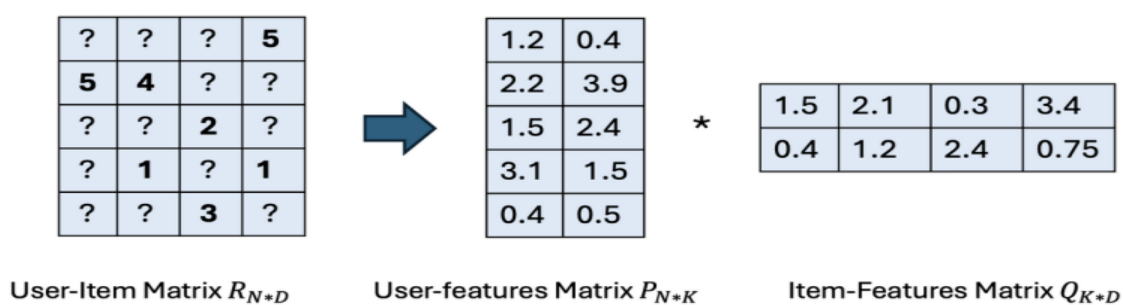
Giờ ta đã có ma trận tương tác để huấn luyện mô hình ở bước tiếp theo cũng như hai tập dữ liệu để kiểm tra mô hình lúc sau.

d. Huấn luyện mô hình Collaborative Filtering theo hướng Matrix Factorization.

Đối với một ma trận tương tác thì việc bị rơi vào tình trạng quá thưa là không hiếm gặp và cũng rất thực tế, ma trận của nhóm có shape là 1000 x 17016 tức xấp xỉ 17 triệu giá trị tuy nhiên từ tập dữ liệu huấn luyện chỉ có 326443 giá trị, khiến cho độ thưa của ma trận - sparsity lên tới 98%. Điều này đồng nghĩa với việc nếu trực tiếp factorization trên ma trận thưa sẽ khó dự đoán được giá trị mang lại ý nghĩa. Vì vậy,

nhóm sẽ tiếp cận mô hình lọc cộng tác theo hướng Matrix Factorization, còn được gọi là kỹ thuật phân rã ma trận.

Có thể hiểu đơn giản là khi ma trận user x item quá thưa, ta có thể tiến hành chia nhỏ thành hai ma trận (user x k) x ma trận (k x item) với k là số lượng thuộc tính ẩn để mô tả cho user và item.



Hình 6: Ví dụ kỹ thuật phân rã ma trận (Nguồn: *What is Matrix Factorization?* - Ali Zonoozi)

Để dễ hình dung hơn, có thể lấy ví dụ trực tiếp từ dự án, ví dụ đây là ma trận tương tác hiện tại:

	T1	T2	T3	...
U1	0.80	0.0	0.6	
U2	0.0	0.9	0.3	
...				

Sau khi phân rã sẽ được hai ma trận nhỏ hơn:

	Thuộc tính ẩn 1	Thuộc tính ẩn 2
U1	0.7	0.2
U2	0.0	0.9

X

	T1	T2	T3
Thuộc tính ẩn 1	0.8	0.1	0.1
Thuộc tính ẩn 2	0.1	0.8	0.1

Tích vô hướng của hai ma trận này chính là ma trận ước lượng được độ tương thích giữa từng user và bài hát kể cả những bài họ chưa từng nghe.

$U1 \mid B = 0.7 \times 0.1 + 0.2 \times 0.8 = 0.23 \rightarrow$ Chưa chắc chắn nhưng cũng có thể U1 sẽ thích bài T2

$U2 \mid A = 0.0 \times 0.8 + 0.9 \times 0.1 = 0.09 \rightarrow$ U2 rất có khả năng sẽ không thích bài T1

Điều đặc biệt ở kỹ thuật này là các thuộc tính ẩn được tự động học từ dữ liệu tương tác mà không cần được gán nhãn rõ ràng - nghĩa là mô hình tự phát hiện ra rằng “một nhóm bài hát có xu hướng được nghe bởi cùng nhóm người”, từ đó rút ra đặc điểm ngầm mô tả cả bài hát lẫn người dùng.

Dựa vào lý thuyết trên, chúng ta sẽ trở lại phân rã `interaction_matrix` của dự án với $k = 50$ thuộc tính ẩn (latent factors).

```
R = interaction_matrix.values.astype(np.float64)
R_boosted = R * 1.2
R_boosted = np.clip(R_boosted, 0, 1)
user_ids = interaction_matrix.index.values
track_ids = interaction_matrix.columns.values
user_mean = np.mean(R_boosted, axis=1)
R_demeaned = R_boosted - user_mean.reshape(-1, 1)
noise = np.random.normal(0, 0.02, R_demeaned.shape)
R_demeaned += noise
```

Đầu tiên ta nhân ma trận tương tác với hệ số 1.2 nhằm tăng độ nhận diện của các tín hiệu dương cũng như trừ đi trung bình (de-mean) của từng user. Đây là một bước rất quan trọng giúp mô hình học tốt hơn, nhớ lại về ma trận tương tác gốc, giá trị strength được tính dựa trên tần suất nghe, tuy nhiên điều này sẽ gây ra hiện tượng “thiên vị” trong mô hình. Có thể hình dung đơn giản là A và B đều cùng gu nhạc, nhưng người dùng A nghe nhạc thường xuyên còn người dùng B lâu lâu mới nghe nhạc, đồng nghĩa các chỉ số strength của A sẽ thường cao hơn B. Nếu không trừ đi trung bình theo từng người dùng thì mô hình có thể nhiễu bởi tần suất nghe và cho rằng A thích bài C hơn B thay vì cả A và B đều thích bài C.

```

min_dim = min(R_demeaned.shape)
k = min(n_factors, min_dim - 1) if min_dim > 1 else 1
U, sigma, Vt = svds(R_demeaned, k=k)
sigma = np.diag(sigma)
R_pred = np.dot(np.dot(U, sigma), Vt) + user_mean.reshape(-1, 1)

```

Sau đó ta sẽ tiến hành bước phân rã ma trận sử dụng phương pháp Singular Value Decomposition (SVD), mục tiêu là phân rã ma trận tương tác sau xử lý $R_demeaned$ thành ba thành phần: $R_demeaned \sim U\Sigma Vt$. Trong đó:

- U: biểu diễn ma trận đặc trưng ẩn của người dùng (user embeddings).
- Σ : là ma trận đường chéo cho biết mức độ quan trọng của từng thuộc tính ẩn.
- Vt: biểu diễn ma trận đặc trưng ẩn của bài hát (item embeddings).

```

R_pred = np.dot(np.dot(U, sigma), Vt) + user_mean.reshape(-1, 1)
R_pred = np.clip(R_pred, 0, None)
if R_pred.max() > 0:
    R_pred = (R_pred / R_pred.max()) * 0.8 + 0.2

```

Cuối cùng, chúng ta kết hợp ba ma trận thành phần U, Σ và Vt, đồng thời khôi phục lại giá trị trung bình của từng người dùng để tái tạo ma trận dự đoán hoàn chỉnh R_pred , chuẩn hóa và lưu mô hình lại dưới dạng file .pkl bao gồm ma trận dự đoán R_pred cho tất cả cặp user - track, cùng với các ma trận embedding U (user) và Vt (item), giá trị kỳ dị sigma, danh sách ID người dùng và bài hát, bộ ánh xạ index, ma trận tương tác gốc và trung bình của từng user $user_mean$. Mỗi phần tử $R_pred[i,j]$ trong ma trận kết quả chính là mức độ tương tác dự kiến giữa người dùng thứ i và bài hát thứ j. Bây giờ khi ta có giá trị $R_pred[3,5] = 0.85$, điều này có nghĩa là người dùng thứ 3 có mức độ tương tác dự kiến là 0.85 đối với bài hát thứ 5, được đánh giá dựa vào đặc trưng âm nhạc và hành vi tương tác học từ dữ liệu huấn luyện.

e. Tải mô hình vào dự án và sử dụng

Để sử dụng mô hình, đầu tiên ta sẽ tiến hành load mô hình vừa mới huấn luyện và bảng `master_features` chứa các thuộc tính của bài hát vào dự án.

```
with open(master_feature_path, 'rb') as f:
    master_features = pickle.load(f)
with open(mf_model_path, 'rb') as f:
    mf_model = pickle.load(f)
```

Toàn bộ các luồng tính toán để có được đầu ra cuối cùng sẽ được tích hợp trong hàm `get_genre_recommendations()`, đây cũng là hàm duy nhất được gọi trong tầng giao diện để trả về danh sách kết quả và hiển thị.

```
def get_genre_recommendations(db, user_id, limit_per_genre=10):
    """Hàm chính lấy recommendations"""
    try:
        user_data = get_cached_user_data(db, user_id)
        purchased_tracks = user_data['purchased_tracks']
        purchased_artists = user_data['purchased_artists']
        is_cold_start = user_data['is_cold_start']
        if is_cold_start:
            result = get_cold_start_recommendations(db, user_id, purchased_tracks,
limit_per_genre)
        else:
            top_genres = get_user_top_genres(db, user_id)
            all_recommendations = []
            for genre in top_genres:
                genre_tracks = recommend_tracks_for_genre(
                    user_id, genre, purchased_tracks, purchased_artists, limit_per_genre)
                all_recommendations.append({
                    'genre': genre,
                    'tracks': genre_tracks,
                    'track_count': len(genre_tracks)
                })
            result = {
```

```

        'user_id': user_id,
        'user_type': 'regular_user',
        'top_genres': top_genres,
        'recommendations': all_recommendations,
        'total_recommended_tracks': sum(len(rec['tracks']) for rec in
all_recommendations)
    }
    clear_user_cache(user_id)
    return result
except Exception as e:
    print(f"Lỗi trong get_genre_recommendations: {e}")
    clear_user_cache(user_id)
    return {'user_id': user_id, 'user_type': 'error', 'recommendations': []}

```

Bước đầu tiên khi sau khi hàm được gọi là truy xuất toàn bộ thông tin của `user_id` được truyền vào bao gồm danh sách các bài đã mua, các nghệ sĩ đã tương tác và kết quả xác định user có rơi vào trường hợp cold start hay không.

Trường hợp 1: Người dùng thuộc cold start

Ở mô hình này cold start được quy định là các user sở hữu ít hơn 5 bài (`'is_cold_start': len(purchased_tracks) < 5`). Hệ thống sẽ tiến hành lọc ra 4 thể loại có nhiều bài nhất trong bảng `master_features` và lấy 10 bài có điểm popularity score cao nhất làm kết quả trả về cuối cùng, giúp người dùng dễ tiếp cận và hình thành sở thích âm nhạc từ từ.

Trường hợp 2: Người dùng không thuộc cold start

Đối với người dùng có `'is_cold_start': False`, hệ thống sẽ đưa ra gợi ý dựa trên các tương tác của họ. Đầu tiên vẫn là bước xác định 4 thể loại nhưng thay vì dựa trên dữ liệu toàn hệ thống thì ta lọc 50 bài gần nhất tra cứu từ bảng `user_history`, danh sách sẽ được sắp xếp dựa trên thể loại xuất hiện nhiều nhất, nếu người dùng nghe ít hơn 4

thể loại hay lịch sử nghe chưa vượt 50 bài thì hàm vẫn hoạt động bình thường và trả về danh sách các thể loại người dùng đã nghe.

Bước tiếp theo, ta sẽ chạy vòng lặp theo từng thể loại để chọn ra 10 bài cho từng thể loại sử dụng hàm `recommend_tracks_for_genre()`

```
available_tracks = master_features[
    (master_features['primaryGenreName'] == genre) &
    (~master_features['trackId'].isin(purchased_tracks))]
```

Vì là gợi ý những bài chưa mua nên trước hết phải tìm các bài thuộc thể loại được truyền vào và loại những bài thuộc `purchased_tracks`. Nhưng đối với các thể loại có tới vài nghìn bài thì chỉ lọc như vậy có thể chưa mang đến hiệu quả cao cho kết quả cuối cùng nên ta sẽ tiếp tục lọc ra một danh sách bài hát, ở đây gọi là `candidate pool`, có độ tương đồng với các bài hát mà người dùng đã mua sử dụng `cosine similarity`.

```
feature_columns = ['trackTimeMillis', 'recency_score']
purchased_features = purchased_tracks_features[feature_columns].fillna(0)
candidate_features = candidate_tracks[feature_columns].fillna(0)
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
all_features = pd.concat([purchased_features, candidate_features])
scaler.fit(all_features)
purchased_scaled = scaler.transform(purchased_features)
candidate_scaled = scaler.transform(candidate_features)
similarities = cosine_similarity(purchased_scaled, candidate_scaled)
avg_similarities = similarities.max(axis=0)
candidate_tracks = candidate_tracks.copy()
candidate_tracks['similarity'] = avg_similarities
return candidate_tracks.nlargest(top_k, 'similarity')
```

Các đặc trưng được chọn để so sánh gồm `recency_score`, `unique_listeners` và `favorite_count`. Dữ liệu của hai nhóm bài hát được chuẩn hóa bằng `StandardScaler` để

đưa về cùng thang đo, tránh một thuộc tính có giá trị lớn chi phối kết quả. Sau đó tính toán độ tương đồng bằng hàm `cosine_similarity()` của `sklearn`, kết quả là một ma trận có kích thước (m, n) - trong đó m là số bài người dùng đã nghe và n là số bài đã lọc ở lần 1. Cuối cùng, hệ thống lấy giá trị trung bình theo cột để thu được điểm `avg_similarity` cho candidate tracks. Những bài hát có điểm trung bình cao nhất sẽ được chọn ra bằng `nlargest(top_k, 'similarity')`.

Sau khi lọc được nhóm bài hát từ bước trên, hệ thống sẽ tiến hành tính điểm cho từng bài hát bằng hàm `calculate_track_score()`, đây sẽ là `final_score` kết hợp từ Matrix Factorization - Collaborative Filtering, Similarity - Content-based Filtering và độ nhiễu.

```
mf_score = 0
if mf_model and str(user_id) in mf_model['user_idx_map'] and track_id_str in
mf_model['track_idx_map']:
    u_idx = mf_model['user_idx_map'][str(user_id)]
    i_idx = mf_model['track_idx_map'][track_id_str]
    mf_score = float(mf_model['R_pred'][u_idx, i_idx])
    has_mf_score = True
elif mf_model and track_id_str in mf_model['track_idx_map']:
    # Warm start: dùng average score của track từ tất cả users
    i_idx = mf_model['track_idx_map'][track_id_str]
    track_scores = mf_model['R_pred'][:, i_idx]
    mf_score = float(np.mean(track_scores[track_scores > 0]))
    mf_score *= 0.7
content_based_score = track.get('similarity', 0)
```

Trước hết, điểm `mf_score` được lấy từ ma trận dự đoán `R_pred` trong mô hình, thể hiện mức độ tương thích giữa người dùng và đặc trưng bài hát:

- Nếu người dùng và bài hát đều tồn tại trong mô hình, hệ thống sử dụng trực tiếp giá trị dự đoán từ `R_pred`.

- Trong trường hợp đây là người dùng mới, chưa tồn tại trong mô hình, nhưng đã có lượng tương tác đáng kể thì sẽ được xử lý điểm theo hướng warm start, tức là lấy trung bình điểm dự đoán R_{pred} của bài hát, đồng thời nhân với 0.7 để phản ánh độ tin cậy thấp hơn.

Còn điểm content-based score chính là giá trị similarity tính bằng cosine similarity ở bước lọc prefiltered trước đó.

```
if has_mf_score:
    score += 0.50 * mf_score
    score += 0.40 * content_based_score
else:
    score += 0.20 * mf_score
    score += 0.70 * content_based_score
```

Hai điểm thành phần này cũng sẽ được nhân theo trọng số tùy vào người dùng này đã tồn tại trong mô hình hay chưa, nếu đã tồn tại thì yếu tố Collaborative Filtering sẽ được ưu tiên hơn và ngược lại.

Cuối cùng, hệ thống bổ sung các yếu tố cá nhân hóa và trả về điểm cuối cùng cho bài hát. Nếu bài hát thuộc nghệ sĩ mà người dùng từng mua sẽ cộng thêm 0.05 điểm; sau đó thêm một nhiễu nhỏ ngẫu nhiên ($+ 0.05 * \text{random.uniform}(0, 1)$) để tránh việc kết quả bị trùng lặp hoàn toàn giữa các lần gọi ý.

3.3.3.4 Mô hình đề xuất giao diện xoay đĩa

- a. Chia dữ liệu ra để cho máy học và test dựa theo user

```
#Chia user theo tỷ lệ 80-20
all_users = dataframes['purchase']['userId'].unique()
np.random.seed(42)
train_users = np.random.choice(all_users, size=int(0.8 * len(all_users)),
replace=False)
test_users = np.array([u for u in all_users if u not in train_users])
```

Ở đây đầu tiên ta lấy danh sách toàn bộ tất cả users đang có trong dữ liệu ra, sau đấy chọn ngẫu nhiên 80% user trong đó ra dành cho việc training, 20% còn lại không train cho mô hình để tránh xảy ra tình trạng data leakage mô hình được học trước toàn bộ dữ liệu.

2. TÁCH PURCHASES THEO USER

```
purchase_data = dataframes['purchase'].copy()
train_purchases = purchase_data[purchase_data['userId'].isin(train_users)]
test_purchases = purchase_data[purchase_data['userId'].isin(test_users)]
```

Sau đó tách file “purchase” ra theo users đã được tách ra như bên trên. Chỉ lịch sử mua hàng của user training thì mới được đưa vào cho mô hình học.

3. TÁCH TRACKS - CHỈ tracks được mua bởi train users

```
train_track_ids = set(train_purchases['trackId'])
test_track_ids = set(test_purchases['trackId'])
train_tracks =
dataframes['tracks'][dataframes['tracks']['trackId'].isin(train_track_ids)].copy()
```

4. TÁCH CÁC DATA KHÁC THEO TRAIN USERS/TRACKS

```
train_mood_data = dataframes['song_mood_community'][
    dataframes['song_mood_community']['trackId'].isin(train_track_ids)
].copy()
train_ratings = dataframes['user_rating'][
    dataframes['user_rating']['userId'].isin(train_users) # QUAN TRỌNG: theo
user
].copy()
```

Tách các dữ liệu khác như là bài hát (tracks), moods, rating của user chỉ được lấy từ user training, ngoài ra collaborative filtering cũng chỉ được train dựa trên data của train users.

STRICT USER-BASED SPLIT

=====

Users: Train=800, Test=200

STRICT SPLIT STATS:

Train purchases: 327,573

Test purchases: 81,470

Train tracks: 17,032

Test tracks: 14,617

Train mood data: 15,517

Train ratings: 15,996

Track overlap: 14,445 tracks

User overlap: 0 users

Hình 7: Cấu trúc dữ liệu sau khi tách để train model

Kết quả chạy ra cho thấy dữ liệu đã được chia ra đầy đủ, và có các track trùng ở cả train và test vì một user có thể mua nhiều bài nên có thể trùng nhau, nhưng giữa 2 bộ dữ liệu thì không có user nào trùng lặp.

b. Các hàm sử dụng để huấn luyện mô hình

Hàm `get_content_candidate_full` dùng để lọc từ dữ liệu ra các bài hát tương tự với bài đang được phát (cùng thể loại, cùng nghệ sĩ) và hàm này sẽ lọc ra 300 bài tiềm năng.

```
def get_content_candidates_full(current_track_id, num_candidates=300):
    current_track = tracks_dict.get(current_track_id)
    if not current_track:
        return []
    current_artist = current_track['artistName']
    current_genre = current_track['primaryGenreName']
```

Đoạn này dùng để lấy được tên nghệ sĩ cùng với thể loại của bài hát mà user đang phát

```
# Tìm bài CÙNG GENRE từ TRAIN data
```

```

mandopop_genres = ['国语流行', 'Mandopop', '中文流行', 'Chinese Pop']
if current_genre in mandopop_genres:
    same_genre = tracks_full[
        (tracks_full['primaryGenreName'].isin(mandopop_genres)) &
        (tracks_full['artistName'] != current_artist)
    ][['trackId']].tolist()
    genre_label = "Mandopop"
else:
    same_genre = tracks_full[
        (tracks_full['primaryGenreName'] == current_genre) &
        (tracks_full['artistName'] != current_artist)
    ][['trackId']].tolist()
    genre_label = current_genre

```

Tìm các bài có cùng genre với bài đang phát, riêng “Mandopop” vì trong dữ liệu có nhiều tên gọi nên gom lại thành một nhóm. Nếu genre hiện tại là Mandopop thì sẽ tìm tất cả các bài thuộc nhóm đó, còn các genre khác thì sẽ tìm theo tên bình thường.

```

artist_count = min(int(num_candidates * 0.6), len(same_artist))
genre_count = min(num_candidates - artist_count, len(same_genre))
candidates = same_artist[:artist_count] + same_genre[genre_count:]

```

Phân bố tỷ lệ lấy bài theo cùng Artist 60%, cùng genre 40%.

```

def get_mood_candidates_fast(user_mood, num_candidates=150):
    """Mood CHỈ từ train data"""
    mood_id = {"Happy": 1, "Sad": 2, "Neutral": 3, "Intense": 4}[user_mood]
    mood_scores = []
    for track_id, data in mood_comm_dict.items():
        if 'mood_weights_fixed' in data:
            mood_score = data['mood_weights_fixed'].get(mood_id, 0)
            if mood_score > 0.1 and track_id in tracks_dict:
                mood_scores.append((track_id, mood_score))

```

```
mood_scores.sort(key=lambda x: x[1], reverse=True)
return [track_id for track_id, score in mood_scores[:num_candidates]]
```

Hàm trên dùng để lấy các candidates dựa theo mood mà người dùng đã chọn ở bên ngoài, đầu tiên sẽ tính điểm mood các bài từ data train mà nhóm đã giả lập người dùng gán mood cho bài hát theo dạng dict (vdu “{1:0.4, 2:0.2, 3:0.1, 4:0.3}”), sau đó lọc ra các bài mood phù hợp với mood đang được chọn.

```
def calculate_mood_score_fast(track_id, user_mood):
    mood_id = {"Happy": 1, "Sad": 2, "Neutral": 3, "Intense": 4}[user_mood]
    mood_data = mood_comm_dict.get(track_id, {})
    raw_score = mood_data.get('mood_weights_fixed', {}).get(mood_id, 0.0)
    if raw_score > 0.7:
        return 0.6
    elif raw_score > 0.5:
        return 0.4
    elif raw_score > 0.3:
        return 0.3
    elif raw_score > 0.1:
        return 0.2
    else:
        return 0.1
```

Hàm này chuẩn hóa điểm mood lại, tránh một số trường hợp một số bài có điểm mood quá cao vdu bài “Thiên Lý Oi” có điểm happy là 0.9 sẽ quá chiếm ưu thế với các bài khác và sẽ luôn được đề xuất nếu ở mood Happy.

```
def calculate_rating_score_fixed(track_id):
    """Rating scoring với normalization đúng"""
    rating = track_ratings.get(track_id, 0)
    if rating == 0:
        return 0.0
    return (rating - 1) / 4.0
```

Hàm này tính điểm dựa trên dữ liệu đánh giá của người dùng, sau đây chuẩn hóa từ thang 0-5 về 0-1

```
def cold_start_recommendations_fixed(current_track_id, user_mood, limit=8):
    print("COLD START: ")
    start_time = time.time()
    current_track = tracks_dict.get(current_track_id)
    # 1. Lấy candidates
    candidates = set()
    if current_track:
        current_artist = current_track['artistName']
        current_genre = current_track['primaryGenreName']
    # Bài cùng artist (nếu có)
    same_artist = tracks_full[
        (tracks_full['artistName'] == current_artist) &
        (tracks_full['trackId'] != current_track_id) &
        (tracks_full['trackId'].isin(tracks_dict.keys())) # CHỈ lấy tracks có tồn tại
    ][['trackId']].tolist()[:3]
    candidates.update(same_artist)
```

Hàm cold start dành cho trường hợp đó là user mới chưa có dữ liệu từ người dùng

```
if current_track:
    current_artist = current_track['artistName']
    current_genre = current_track['primaryGenreName']
    # Bài cùng artist (nếu có)
    same_artist = tracks_full[
        (tracks_full['artistName'] == current_artist) &
        (tracks_full['trackId'] != current_track_id) &
        (tracks_full['trackId'].isin(tracks_dict.keys())) # CHỈ lấy tracks có tồn tại
    ][['trackId']].tolist()[:3]
```

```

candidates.update(same_artist)
# Bài cùng genre (nếu có)
same_genre = tracks_full[
    (tracks_full['primaryGenreName'] == current_genre) &
    (tracks_full['artistName'] != current_artist) &
    (tracks_full['trackId'].isin(tracks_dict.keys())) # CHỈ lấy tracks có tồn tại
]['trackId'].tolist()[:5]
candidates.update(same_genre)

```

Dựa vào bài đang phát lấy thông tin về nghệ sĩ, và thể loại sau đó như cũ xài content để lấy các bài có cùng thể loại và nghệ sĩ, tuy nhiên đây là người dùng mới, nhằm hiệu quả hơn thì chỉ lấy lần lượt 3 bài cho cùng nghệ sĩ và 5 bài cho cùng thể loại vì người dùng mới có thể sẽ có nhu cầu mua nhiều thể loại nhạc, hoặc là nhạc của nhiều nghệ sĩ khác nhau.

Lấy bài hát phổ biến từ các thể loại nhạc

```

def get_truly_diverse_popular_tracks(num_tracks=80):
    """Lấy bài phổ biến từ NHIỀU genre"""
    diverse_tracks = []
    # Ưu tiên các genre đa dạng (giảm Mandopop)
    target_genres = ['Pop', 'K-Pop', 'R&B/Soul', 'Hip-Hop/Rap', 'Dance', 'J-Pop',
'Alternative', 'Rock', 'Electronic']
    for genre in target_genres:
        genre_tracks = tracks_full[tracks_full['primaryGenreName'] == genre]
        if len(genre_tracks) > 0:
            # Lấy top tracks của genre này - CHỈ lấy tracks có trong tracks_dict VÀ
track_popularity
            genre_track_ids = [
                tid for tid in genre_tracks['trackId']
                if tid in tracks_dict and tid in track_popularity.index
            ]

```

```

        if genre_track_ids:
            try:
                genre_popular = track_popularity[genre_track_ids].nlargest(8)
                diverse_tracks.extend(genre_popular.index.tolist())
            except KeyError:
                # Nếu có lỗi, bỏ qua genre này
                continue

mandopop_tracks = tracks_full[tracks_full['primaryGenreName'].isin(['Mandopop',
'国语流行', '中文流行'])]

if len(mandopop_tracks) > 0:
    mandopop_ids = [
        tid for tid in mandopop_tracks['trackId']
        if tid in tracks_dict and tid in track_popularity.index
    ]
    if mandopop_ids:
        try:
            mandopop_popular = track_popularity[mandopop_ids].nlargest(5)
            diverse_tracks.extend(mandopop_popular.index.tolist())
        except KeyError:
            pass

```

Hàm này giúp chặn việc mô hình có thể sẽ recommend quá tập trung vào một thể loại trước tiên list target_genres là các genres nổi tiếng sẽ được ưu tiên, mỗi genres lấy 8 bài, riêng “Mandopop” do có các ký tự tượng hình nên được xử lý riêng để có thể phân biệt. Sau khi chạy xong sẽ trả về mỗi genres 8 bài nổi tiếng, còn riêng Mandopop chỉ lấy 5 bài tránh việc nếu đang phát bài tiếng Trung phần rec sẽ bị không đa dạng.

```

# Bài có mood phù hợp từ NHIỀU GENRE
mood_tracks = get_mood_candidates_fast(user_mood, 100)
candidates.update(mood_tracks)

# Bài phổ biến từ CÁC GENRE ĐA DẠNG
diverse_popular = get_truly_diverse_popular_tracks(80)

```



```
candidates.update(diverse_popular)
```

Tiếp theo xài lại các hàm bên trên tiếp tục lấy thêm 100 bài có điểm mood phù hợp với mood mà user đang sử dụng và lấy từ đa dạng các thể loại nhạc. Tương tự cũng sẽ lấy 80 bài có độ nổi tiếng cao từ các thể loại khác nhau.

2. Tính điểm với TRỌNG SỐ

```
scored_tracks = []
for track_id in candidates:
    # QUAN TRỌNG: Kiểm tra track có tồn tại
    if track_id not in tracks_dict:
        continue
    track = tracks_dict[track_id]
    # Điểm đơn giản
    content_score = 0.0
    if current_track:
        if track['artistName'] == current_track['artistName']:
            content_score = 1.0
        elif track['primaryGenreName'] == current_track['primaryGenreName']:
            content_score = 0.3
    mood_score = calculate_mood_score_fast(track_id, user_mood)
    popularity_score = track_popularity.get(track_id, 0) / max_popularity if
max_popularity > 0 else 0.0
```

Điểm mood, với điểm popularity thì vẫn tính bth như trước, riêng điểm content thì đơn giản hóa hơn, với trường hợp user cũ, ưu tiên cho các bài hát có cùng nghệ sĩ đơn giản hơn với user mới, ví dụ như bạn thích nghe nhạc Jack thì sẽ ưu tiên đề xuất một số bài khác của Jack.

TRỌNG SỐ: Mood quan trọng nhất

```
total_score = (
    content_score * 0.2 +
    mood_score * 0.7 +
```

```
popularity_score * 0.1 -  
mandopop_penalty  
)
```

Đối với user cold start trọng số mood sẽ quan trọng nhất vì đó là cái thu thập được trực tiếp của user khi đã đăng nhập vào ứng dụng, tiếp đó content để đảm bảo rằng vẫn sẽ có cái bài cùng chủ đề với các bài mà user đang nghe, đặc biệt với thể loại nhạc trung đặc biệt của tiếng tượng hình nên khi người dùng chọn, mô hình sẽ đề xuất toàn bộ thì đều nhạc trung nhằm đa dạng hóa đề xuất ban đầu cho user nên thêm một biến trừ mandopop vào điểm tổng nhằm mục đích nếu có quá nhiều bài nhạc trung điểm cao rồi thì sẽ trừ bớt điểm đi để các bài thể loại khác có thể được đề xuất.

3. Đa dạng hóa - giới hạn mỗi genre

```
scored_tracks.sort(key=lambda x: x['total_score'], reverse=True)  
final_recommendations = []  
artist_count = {}  
genre_count = {}  
for track in scored_tracks:  
    artist = track['artistName']  
    genre = track['genre']  
    # Giới hạn: max 2 bài/artist, max 2 bài/genre (trừ genre chính)  
    artist_ok = artist_count.get(artist, 0) < 2  
    genre_ok = genre_count.get(genre, 0) < 2  
    if artist_ok and genre_ok:  
        final_recommendations.append(track)  
        artist_count[artist] = artist_count.get(artist, 0) + 1  
        genre_count[genre] = genre_count.get(genre, 0) + 1  
    if len(final_recommendations) >= limit:  
        break
```

Thêm ràng buộc cho phần đề xuất của cuối cùng để đa dạng đầu ra cho coldstart user thì mỗi artist và mỗi genre tối đa chỉ có 2 bài nên phần đề xuất cho coldstart bao giờ cũng chắc chắn là sẽ có 8 bài hát ít nhất là từ 4 nghệ sĩ và 4 thể loại nhạc khác nhau.

```
def precompute_full_similarity_matrix(track_vectors, top_k=100):
    """Precompute FULL similarity matrix"""
    all_track_ids = list(track_vectors.keys())
    matrix_vectors = np.array([track_vectors[tid] for tid in all_track_ids])
    # Normalize features
    scaler = StandardScaler()
    normalized_vectors = scaler.fit_transform(matrix_vectors)
    # Add small noise
    noise = np.random.normal(0, 0.01, normalized_vectors.shape) # 1% noise
    normalized_vectors = normalized_vectors + noise
    similarity_matrix = cosine_similarity(normalized_vectors)
    # Apply similarity smoothing
    similarity_matrix = np.clip(similarity_matrix, 0.1, 0.9)
```

Cuối cùng khi ứng dụng thật vào app nhóm đã chia mô hình hybrid ra làm 3 tầng và sử dụng trong các trường hợp khác nhau. Đầu tiên nhóm tiến hành pre-train lại model content từ google collab có cosine tổng hợp lại 100 bài phù hợp với mỗi track có trong dữ liệu với các thông tin về bài hát, nghệ sĩ và thể loại nhạc, cuối cùng ở class mô hình trong ứng dụng sẽ tùy vào trường hợp các user mà quyết định áp dụng tầng mô hình nào:

```
#LAYER 1: Pre-trained model
if current_track_id in self.model['similarity_dict']:
    recs = self._get_from_full_matrix(current_track_id, user_mood, limit * 3,
    purchased_tracks)
    if recs:
        #FILTER CUỐI CÙNG
        final_rec = self._force_purchase_filter(recs, purchased_tracks, limit)
```

```

        return final_recs

    # LAYER 2: Real-time similarity
    recs = self._get_realtime_track_similarity(current_track_id, user_mood, limit *
3, purchased_tracks)
    if recs:
        #FILTER CUỐI CÙNG
        final_recs = self._force_purchase_filter(recs, purchased_tracks, limit)
        return final_recs

    #LAYER 3: Smart cold start
    recs = self._get_smart_cold_start(user_id, user_mood, limit * 3,
purchased_tracks)
    #FILTER CUỐI CÙNG
    final_recs = self._force_purchase_filter(recs, purchased_tracks, limit)
    return final_recs

```

Tầng 1: Content-Based sử dụng cho user cũ đã có dữ liệu mua nhạc và bài hát đang nghe nằm trong similarity dict là top 100 bài hát giống với từng track trong dữ liệu.

Tầng 2: Real-time đây chính là mô hình sử dụng content theo thời gian thật, áp dụng cho user cũ đã có dữ liệu nhưng bài hát đang phát không nằm trong similarity dict.

Tầng 3: Đây là tầng Cold-Start dành cho user vừa mới đăng ký vào app hoàn toàn chưa có dữ liệu gì hết, hệ thống sẽ gợi ý các bài nhạc nổi tiếng và được đánh giá cao ở đa dạng nghệ sĩ và các thể loại. Và trong tầng cold-start có sự khác biệt nhỏ nhỏ khi còn một lớp nhỏ trong đó là smart_cold_start query lên collection “purchase” của user để tối ưu mô hình mỗi khi user mới mua thêm một bài mới.

```

# Rank-based score
rank_score = 0.9 - (i * 0.02)
if rank_score < 0.6:
    rank_score = 0.6

```

```

        mood_score = self.model['mood_weights'].get(similar_track_id,
        {}).get(user_mood, 0.1)
        total_score = rank_score * 0.7 + mood_score * 0.3
        if total_score > 0.1:
            track_info = self.get_track_info(similar_track_id)
            if track_info:
                scored_tracks.append({
                    **track_info,
                    'score': total_score,
                    'reason': 'content_based_natural'
                })
            if len(scored_tracks) >= limit * 2: # Lấy nhiều hơn để final filter
                break

```

Ở tầng 1, ta lấy điểm rank từ trong top 100 similarity dict với bài đang phát, rank càng cao điểm sẽ càng cao, sau đây sẽ tính điểm cùng với trọng số điểm mood để ra được final score.

```

score = 0.3 # Base score
    if track.get('primaryGenreName') == current_genre:
        score += 0.3
    elif track.get('primaryGenreName') in related_genres:
        score += 0.2
    if track.get('artistName') == current_artist:
        score += 0.2
    popularity = track.get('popularity', 0) or 0
    score += min(popularity / 200, 0.2)
    artist = track.get('artistName', '')
    genre = track.get('primaryGenreName', '')
    if artist not in selected_artists:
        score += 0.1
    if genre not in selected_genres:

```

```
score += 0.05

mood_score = self.model['mood_weights'].get(track_id,
{}).get(user_mood, 0.1)

score += mood_score * 0.1
```

Ở tầng 2 này khi bài hát không nằm trong similarly dict nên sẽ query trực tiếp vào database để lấy thông tin, với điểm ban đầu là 0.3 sau đây sẽ được cộng dồn nếu như là bài cùng thể loại hoặc cùng nhóm thể loại, cùng nghệ sĩ, ngoài ra cộng thêm một trọng số nhỏ hơn cho các bài có điểm đánh giá cao và điểm mood phù hợp với current mood của user, cuối cùng sẽ xếp hạng các bài hát dựa trên điểm số này.

```
# Phân tích genre từ các bài đã mua
purchased_genres = set()
for track_id in purchased_tracks:
    track = self.tracks_db.find_one(
        {"trackId": track_id},
        {'primaryGenreName': 1}
    )
    if track and track.get('primaryGenreName'):
        purchased_genres.add(track.get('primaryGenreName'))

# Ưu tiên 1: Tracks cùng genre với purchased tracks
if purchased_genres:
    genre_tracks = self.tracks_db.find({
        "primaryGenreName": {"$in": list(purchased_genres)},
        "trackId": {"$nin": list(purchased_tracks)}
    }).limit(limit)
    for track in genre_tracks:
        track_info = self.get_track_info(track['trackId'])
        if track_info:
            candidates.append({
                **track_info,
                'score': 0.8, # High score for same genre
```

```
'reason': 'similar_to_purchased'
}))
```

Ở phần `smart_cold_start` này đầu tiên ưu tiên số 1 lớp này sẽ phân tích lịch sử mua của user, ưu tiên lấy các bài cùng thể loại nhạc với các bài đã được user mua trước đó.

```
# Ưu tiên 2: Mood-based từ pre-trained model
mood_candidates = []
for track_id, weights in list(self.model['mood_weights'].items())[:50]:
    if (track_id not in purchased_tracks and
        weights.get(user_mood, 0) > 0.5):
        track_info = self.get_track_info(track_id)
        if track_info:
            mood_candidates.append({
                **track_info,
                'score': weights.get(user_mood, 0),
                'reason': 'mood_match'
            })
        if len(mood_candidates) >= 10:
            break
candidates.extend(mood_candidates)
```

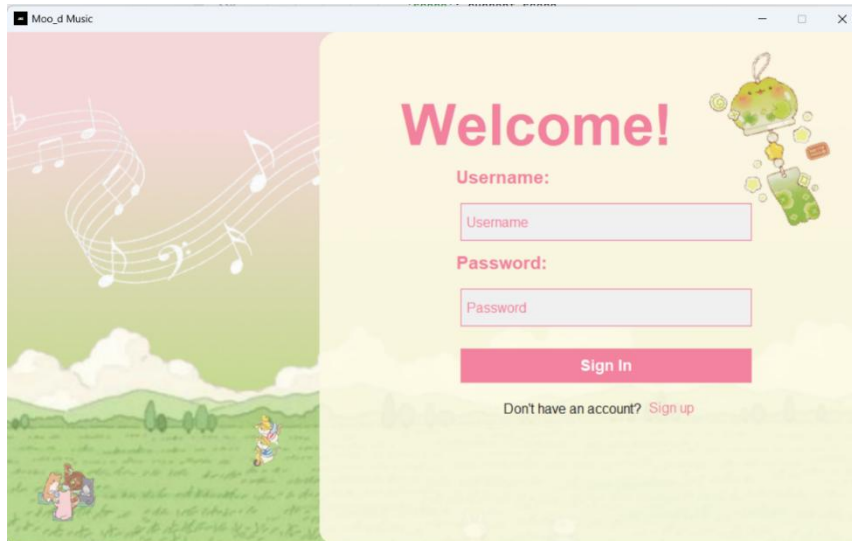
Ưu tiên thứ 2 sẽ lấy thêm các bài có điểm số mood cao phù hợp với current mood của user từ similarity dict của tầng 1, sau cùng nếu vẫn chưa có đủ bài sẽ lấy thêm các bài còn thiếu từ điểm popularity vào. Ở lớp này ưu tiên sự đa dạng cho user mới nên sẽ không lấy bài hát theo kiểu tính điểm mà sẽ lấy các bài cao nhất của tùy tiêu chí ưu tiên sau đó sẽ không lấy từ tiêu chí đó nữa mà sẽ ưu tiên lấy từ tiêu chí khác. Như vậy nếu user vừa mới tạo tài khoản các dữ liệu về người này vẫn sẽ chưa được thêm vào model để pretrained, trong thời gian chờ pre train model theo định kỳ thì cold start vẫn sẽ đảm bảo đề xuất nhạc cho người dùng ổn định dựa vào việc truy xuất vào lịch sử mua của người đó.

CHƯƠNG 4: KẾT QUẢ VÀ PHÂN TÍCH

4.1 Kết quả đạt được

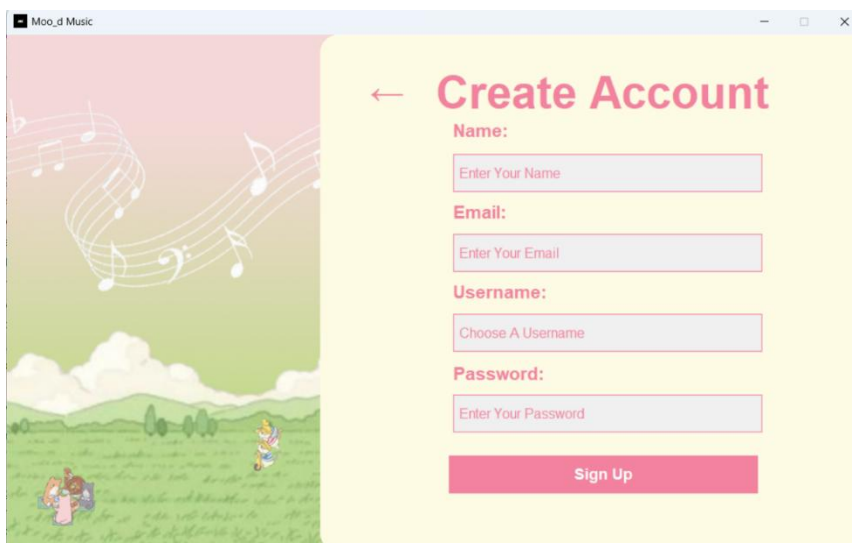
4.1.1 Giao diện đăng ký, đăng nhập tài khoản

Giao diện đầu tiên giao diện đăng nhập, ở đây người dùng có thể đăng nhập vào ứng dụng hay tạo tài khoản mới bằng cách bấm vào chữ ‘Sign up’.



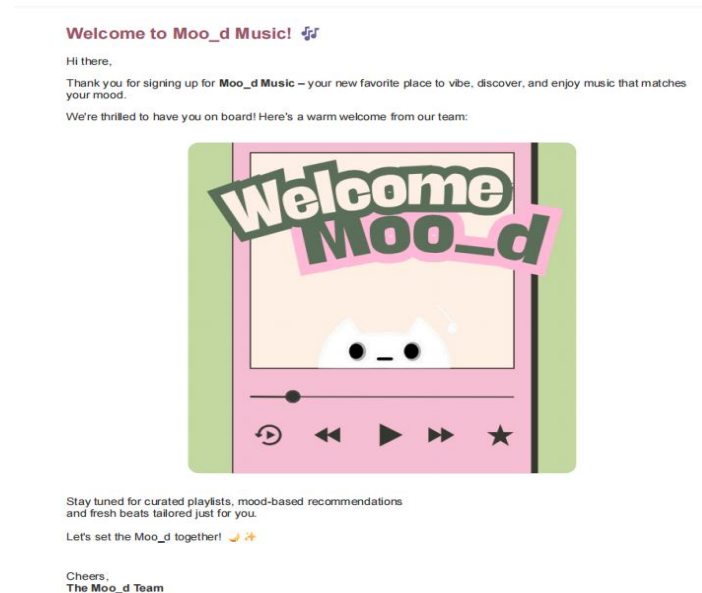
Hình 8: Giao diện đăng nhập

Màn hình đăng ký tài khoản cho phép người dùng nhập vào tên, tên người dùng, mật khẩu cùng với email, tất cả đều có ràng buộc trong trường hợp người dùng nhập thiếu trường hay sai cú pháp, ứng dụng sẽ gửi cảnh báo đến người dùng.



Hình 9: Giao diện đăng ký

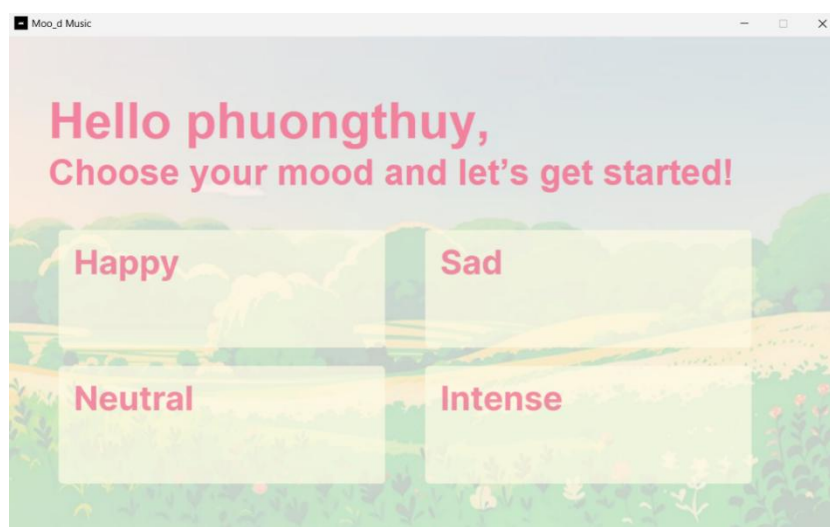
Sau khi đăng ký tài khoản thành công và tài khoản email người dùng sử dụng để đăng ký thật sự tồn tại, người dùng sẽ nhận được một email xác nhận là đã trở thành một thành viên của Moo_d.



Hình 10: Email người dùng mới

4.1.2 Giao diện chọn mood

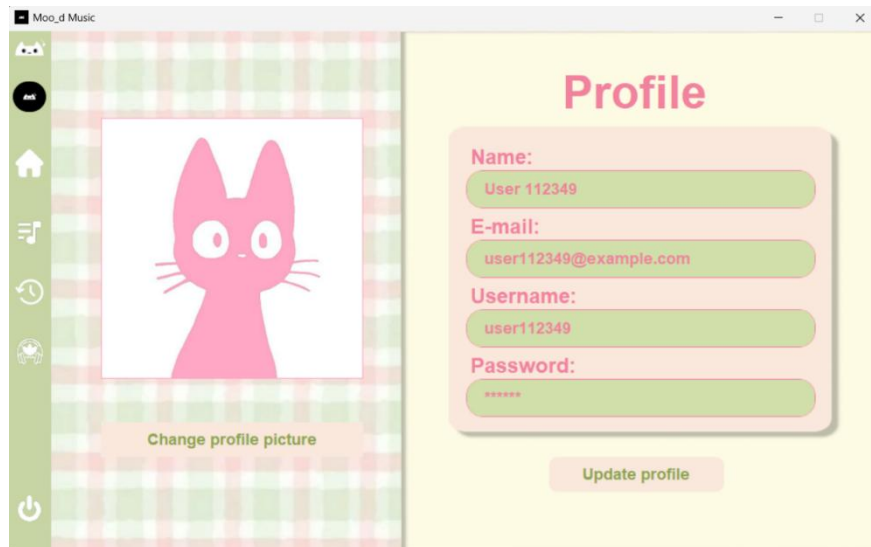
Ở lần cải tiến này, nhóm đã bổ sung thêm 2 mood mới là “Neutral” và “Intense”, thay vì chỉ có hai mood cơ bản là “Happy” và “Sad”. Sự thay đổi này sẽ giúp người dùng có đa dạng sự lựa chọn hơn và tăng cá nhân hóa. Và từ mood này sẽ có những bài hát được đề xuất đúng với tâm trạng của người nghe.



Hình 11: Giao diện chọn mood

4.1.3 Giao diện profile

Giao diện Profile có thể được mở từ nút thứ 2 ở thanh công cụ, ở đây người dùng có thể xem và điều chỉnh thông tin của mình, cũng như cập nhật hình đại diện, nếu là người dùng mới thì ảnh đại diện sẽ là ảnh mặc định của ứng dụng.

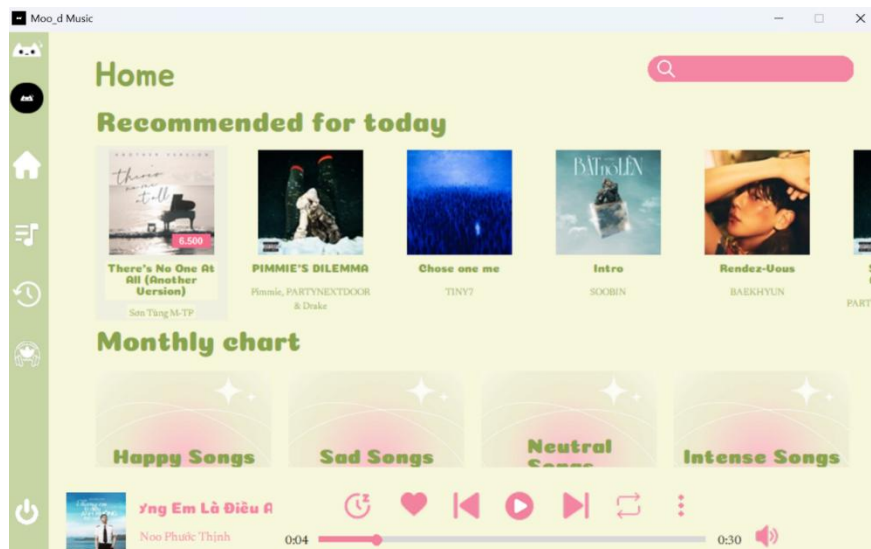


Hình 12: Giao diện profile

4.1.4 Giao diện home và các nút chức năng

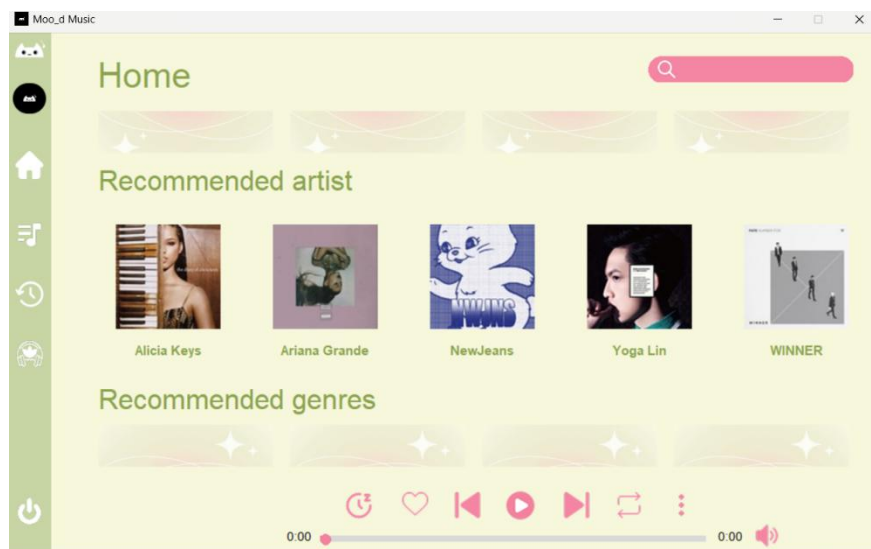
4.1.4.1 Giao diện

Ở giao diện home sẽ hiển thị kết quả của ba mô hình (Recommend for today, Recommend artist, Recommend genres) và một bảng xếp hạng của các bài hát được nghe nhiều nhất trong tháng phân theo từng mood. Đối với Recommend for today, có 10 bài hát sẽ được xuất hiện, vì đều là bài chưa được mua nên khi di chuột vào sẽ xuất hiện giá tiền, người dùng bấm nút là có thể chuyển sang giao diện thanh toán.

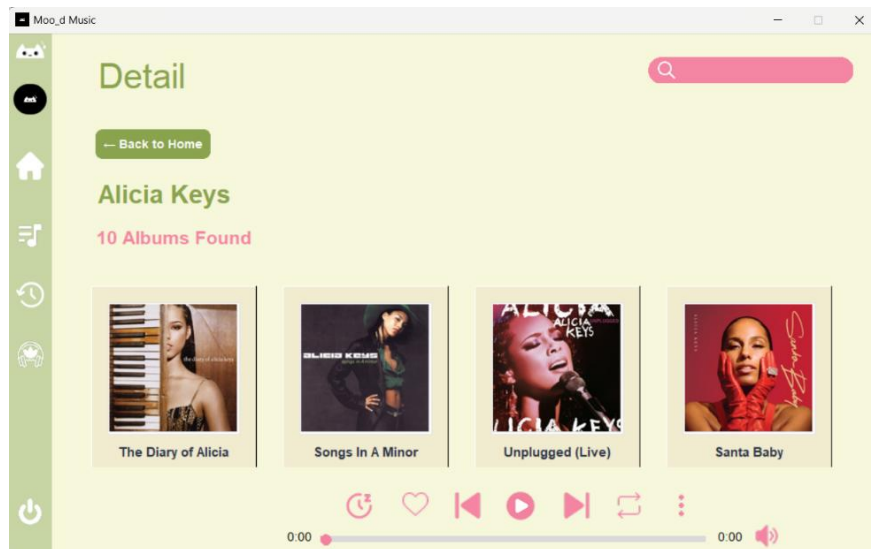


Hình 13: Giao diện mô hình Recommend for today

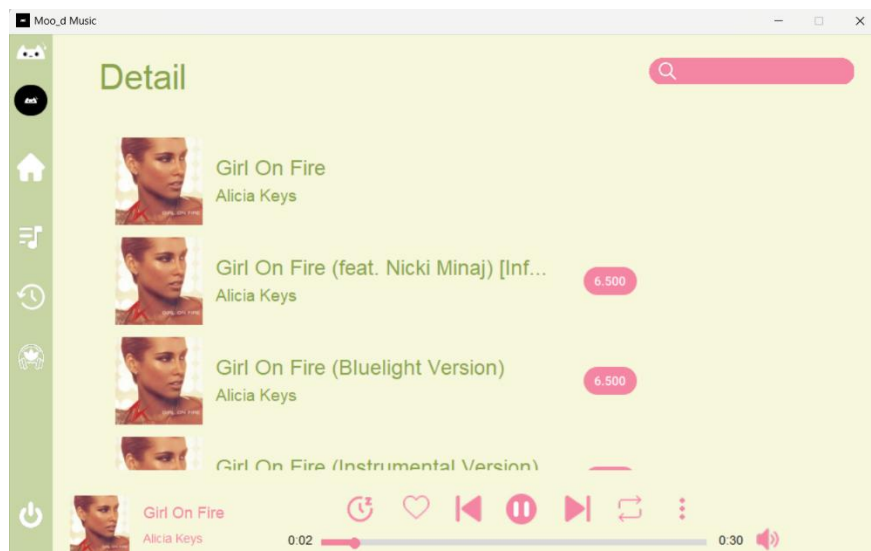
Ở mục Recommended artist, sẽ hiển thị tổng cộng 8 nghệ sĩ cực đúng “gu” của người dùng. Mỗi người dùng sẽ có các nghệ sĩ được gợi ý khác nhau. Khi nhấn vào nghệ sĩ bất kỳ, giao diện sẽ chuyển sang trang Detail - hiển thị chi tiết toàn bộ album của người đó. Ví dụ ở đây khi bấm vào Alicia Keys, ta sẽ thấy tất cả album của cô ấy. Nếu tiếp tục bấm vào album mong muốn, giao diện sẽ hiển thị tất cả bài hát nằm trong album đấy. Bạn có thể lựa chọn mua nhạc đối với những bài chưa sở hữu, nếu đã mua ta bấm trực tiếp vào bài hát để phát nhạc.



Hình 14: Giao diện mô hình Recommend artist



Hình 15: Giao diện album



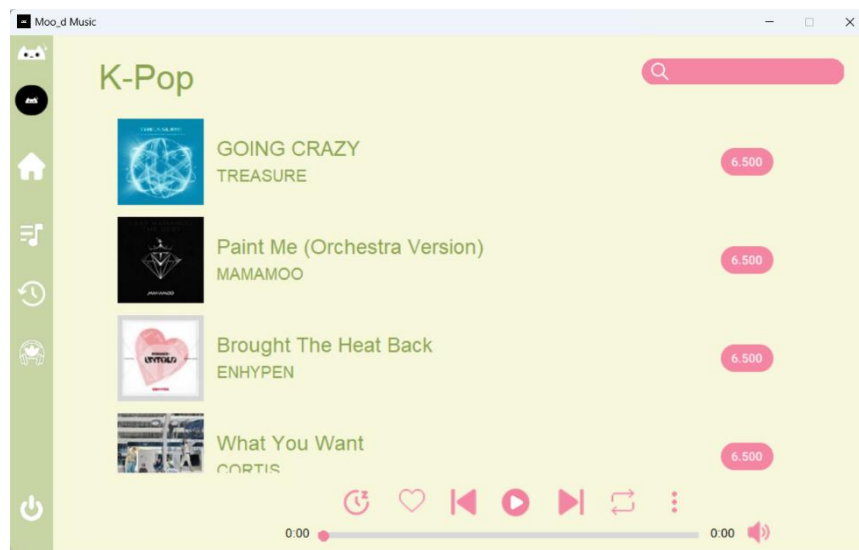
Hình 16: Danh sách bài hát trong album cụ thể

Mô hình cuối cùng ở màn hình chính là mô hình gợi ý nhạc theo thể loại, ở đây sẽ hiển thị tối thiểu 1 và tối đa là 4 thể loại mà người dùng nghe nhiều nhất.



Hình 17: Giao diện mô hình Recommend genres

Người dùng sẽ nhận được 10 gợi ý khi ấn chọn vào thể loại tương ứng, tất cả gợi ý là những bài mà người dùng chưa sở hữu nên cũng có thể mua từ giao diện này.



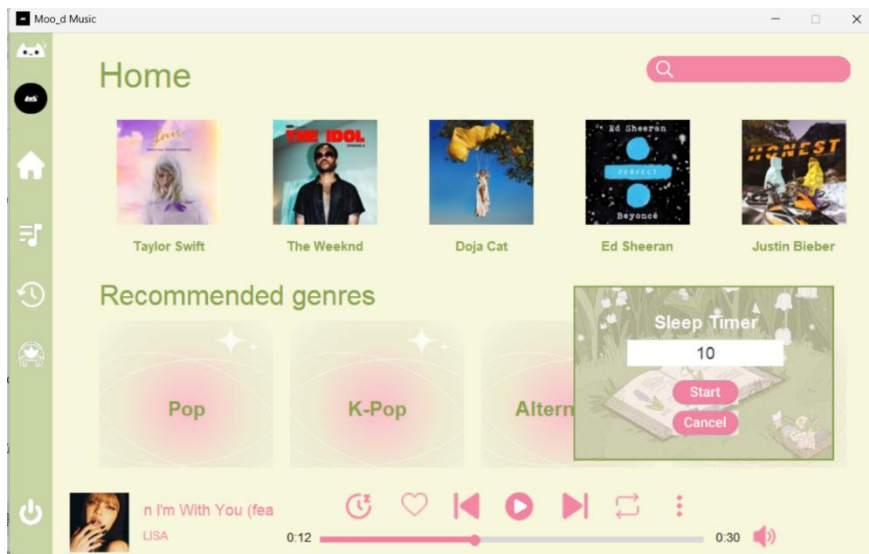
Hình 18: Danh sách bài hát trong genre được gợi ý

4.1.4.2 Nút chức năng

Có 8 nút chức năng giúp người dùng tương tác với nhạc:

- Nút play/pause dùng để bật/dừng nhạc.
- Nút next sẽ chuyển người dùng sang nghe bài kế tiếp (là bài có thời gian mua gần hơn). Nút previous sẽ ngược lại.
- Ở nút lặp bài hát, có ba chế độ: no repeat, repeat one time (lặp bài hát hiện tại 1 lần, sau đó chuyển sang bài khác và cũng lặp bài đó 1 lần), repeat always (lặp liên tục đến khi chuyển nút sang chế độ no repeat).
- Nút ba chấm: đây là nút menu để xổ ra hai lựa chọn của người dùng đối với bài hát là đánh giá bài hát (Rating) và gán cảm xúc cho bài hát (Mood).
- Nút tim sẽ tự kiểm tra xem bài hát có nằm trong danh sách yêu thích không để cập nhật cho đúng, nếu chưa yêu thích thì khi thả tim, bài hát sẽ được thêm bản ghi vào danh sách yêu thích trên mongodb và cập nhật trong giao diện.

- Nút Rate dành cho việc đánh giá bài hát khi nhấn vào người dùng có thể đánh giá cho bài hát với thang đo (1-5) sao. Nhấn Save để lưu kết quả đánh giá, kết quả sẽ được lưu vào database của Moo_d.
- Nút mood giúp cho người dùng đánh giá được bài hát đó đang mang lại cảm xúc như thế nào cho mình, hiện tại ứng dụng đang có 4 mood mặc định, nếu chưa có mood nào được chọn thì sẽ là nút màu trắng khi được chọn 4 mood “Happy”, “Sad”, “Neutral”, “Intense” sẽ có chuyển thành 4 màu tương ứng lên giao diện UI là xanh lá, xanh biển, vàng và hồng. Thông qua đó người dùng biết được bài hát này đang được mình gắn với cảm xúc nào.
- Nút sleep timer: đây là nút cài đặt thời gian để người dùng có thể linh hoạt thiết lập thời gian sử dụng ứng dụng. Sau khi hết giờ, ứng dụng sẽ tự động dừng phát nhạc. Đơn vị được sử dụng ở đây là phút.



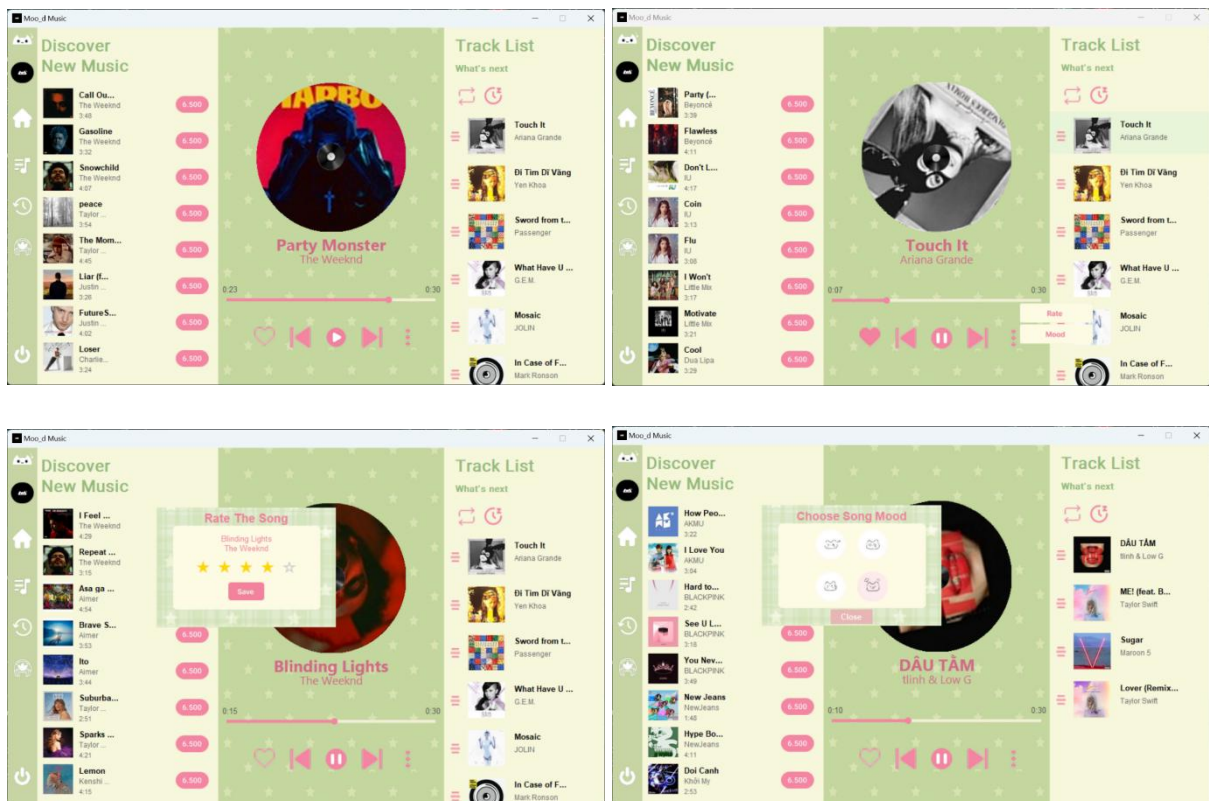
Hình 19: Giao diện chức năng nút sleep timer

4.1.5 Giao diện đĩa nhạc

Đây là giao diện cho từng bài hát, mỗi khi người dùng click vào ảnh bìa bài hát nhỏ nằm ở bên dưới góc trái của giao diện màn hình chính, app sẽ chuyển qua giao diện phát nhạc với chiếc đĩa xoay chứa ảnh bìa của bài hát đó, tên bài hát và nghệ sĩ được cập nhật ở vùng bên dưới của đĩa, khi nhạc được phát đĩa sẽ xoay tạo cảm giác dễ chịu khi nghe nhạc, khi nhạc dừng đĩa cũng dừng theo.

Ở bên phải của giao diện sẽ là Tracklist gồm các bài hát user đã mua, và sẽ phát tiếp sau khi bài hát hiện tại đã phát xong. Lần đầu từ giao diện chính vào Tracklist sẽ chỉ load 6 bài từ các bài người dùng đã mua, khi bài hát ở vị trí thứ 3 từ cuối cùng đếm lên được nhấn vào thì danh sách Tracklist sẽ load và hiển thị thêm 10 bài, điều này sẽ tiếp tục lặp lại cho đến khi hiển thị hết danh sách các bài đã được người dùng mua.

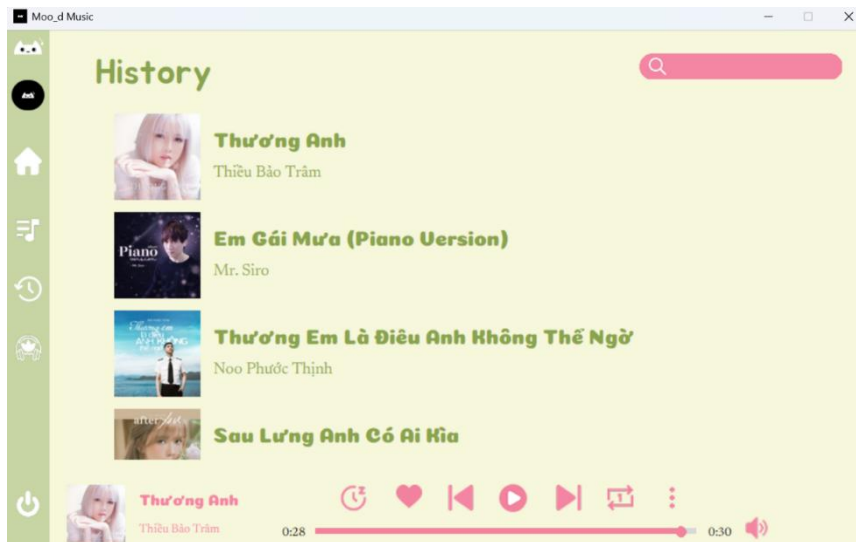
Điều đặc biệt nằm ở bên trái của giao diện này là khu vực Discover sẽ hiển thị 8 bài hát đề xuất từ mô hình đĩa xoay dựa trên bài hát đang phát và cảm xúc hiện tại của người dùng, ngay tại đây nếu click vào nút mua “6.500” người dùng sẽ được chuyển đến giao diện thanh toán để tiến hành giao dịch cho bài hát được bấm vào.



Hình 20: Giao diện đĩa nhạc

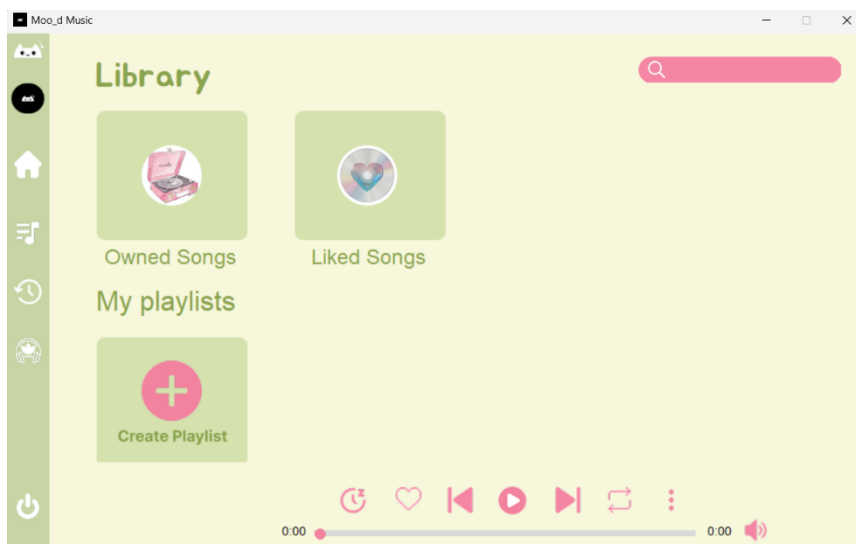
4.1.6 Giao diện lịch sử nghe

Giao diện sẽ hiển thị lịch sử nghe của người dùng dựa trên file “user_history” trên MongoDB. Các bài hát có thời gian nghe gần hơn sẽ xếp phía trên.



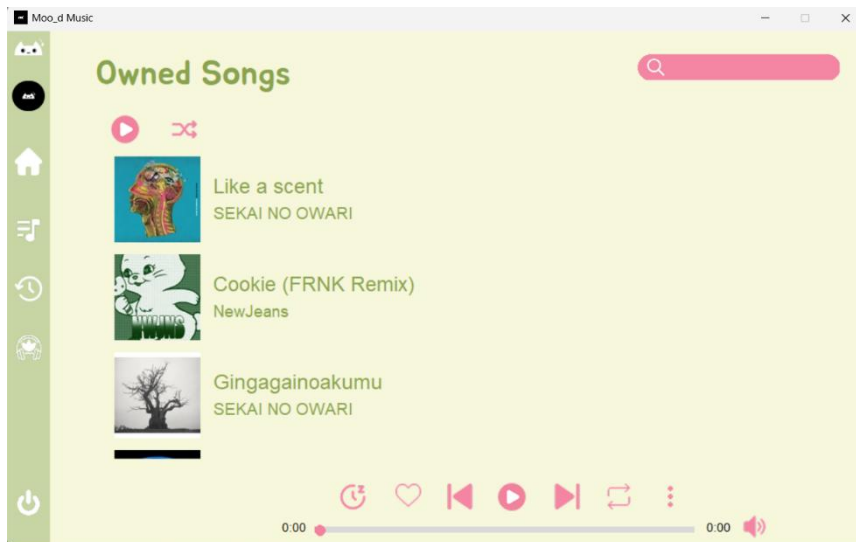
Hình 21: Giao diện lịch sử nghe nhạc

4.1.7 Giao diện thư viện

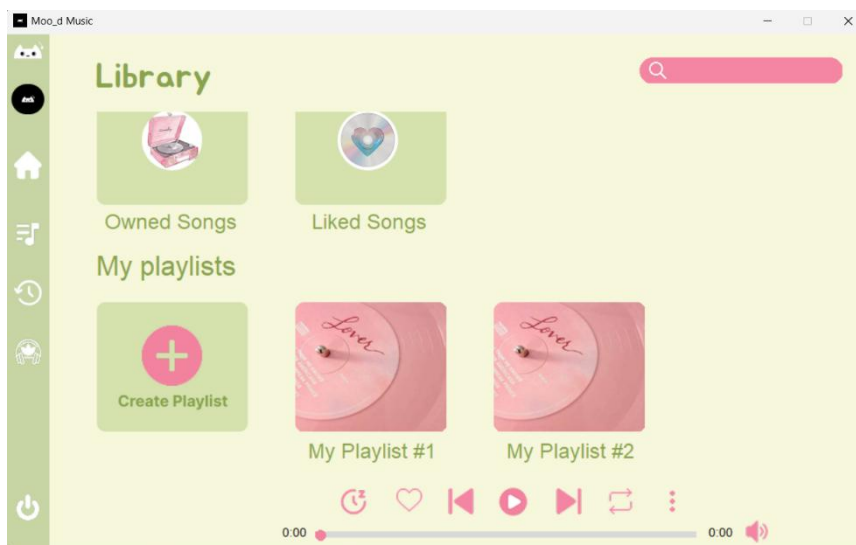


Hình 22: Giao diện thư viện

Giao diện thư viện sẽ hiển thị mặc định 2 playlist là Owned Songs gồm những bài người dùng đã mua lấy dữ liệu từ file “purchase” và Liked Songs gồm những bài người dùng này đã thích lấy dữ liệu từ file “user_favorite” trên MongoDB. Trong hai giao diện này, người dùng có thể phát bất kỳ bài nào mình muốn hoặc phát ngẫu nhiên các bài hát nhờ vào nút play và nút shuffle ở đầu danh sách. Các bài hát trong đây sẽ được hiển thị dựa vào thời gian mua hoặc thích gần đây nhất. Giao diện cụ thể của hai playlist này sẽ được thể hiện qua hình bên dưới.

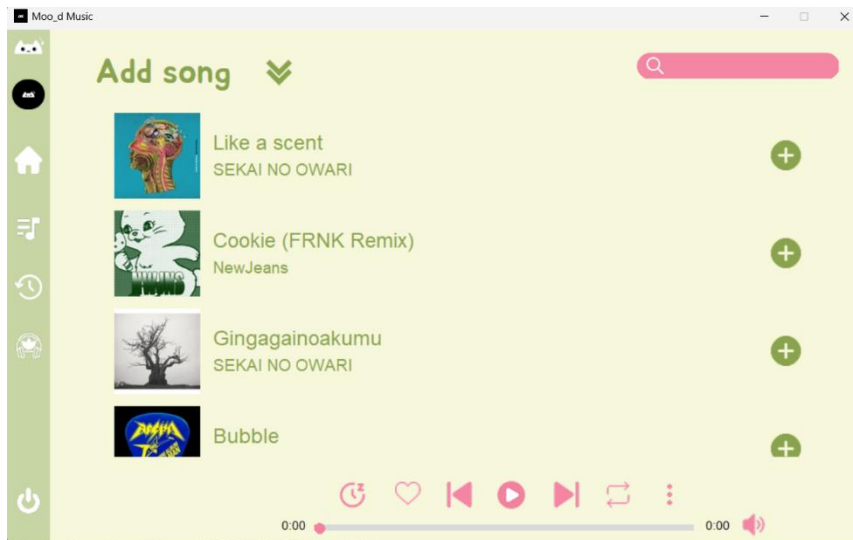


Hình 23: Giao diện bài hát trong Owned Songs



Hình 24: Giao diện khi tạo playlist

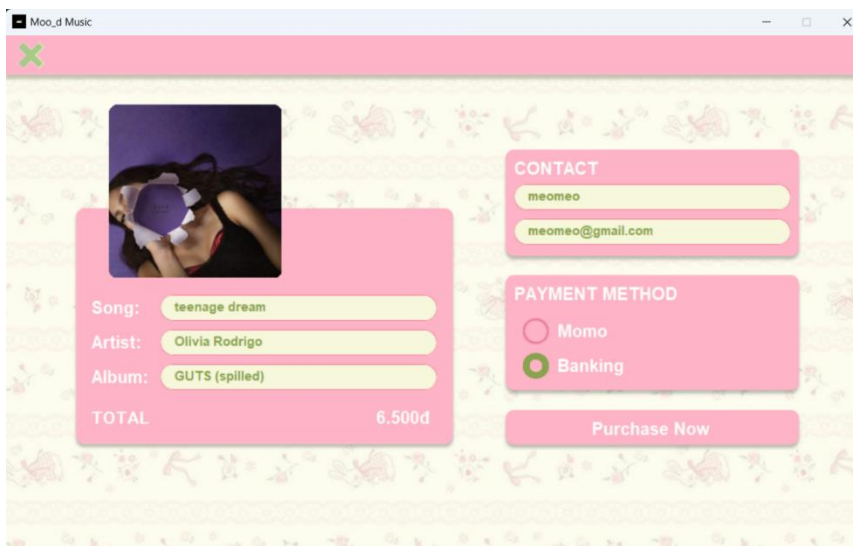
Ngoài ra, trong giao diện thư viện người dùng còn có thể tạo ra các playlist cá nhân hóa nhờ vào chức năng của nút “Create Playlist”. Trong các playlist, giờ đây có thêm chức năng “Add song” cho phép người dùng sẽ có thể thêm bất kì bài hát nào mà mình muốn nhưng chỉ giới hạn trong các bài hát đã mua mà thôi. Về phần các chức năng khác trong ứng dụng giao diện này vẫn sẽ đảm bảo đầy đủ.



Hình 25: Giao diện Add song

4.1.8 Giao diện thanh toán

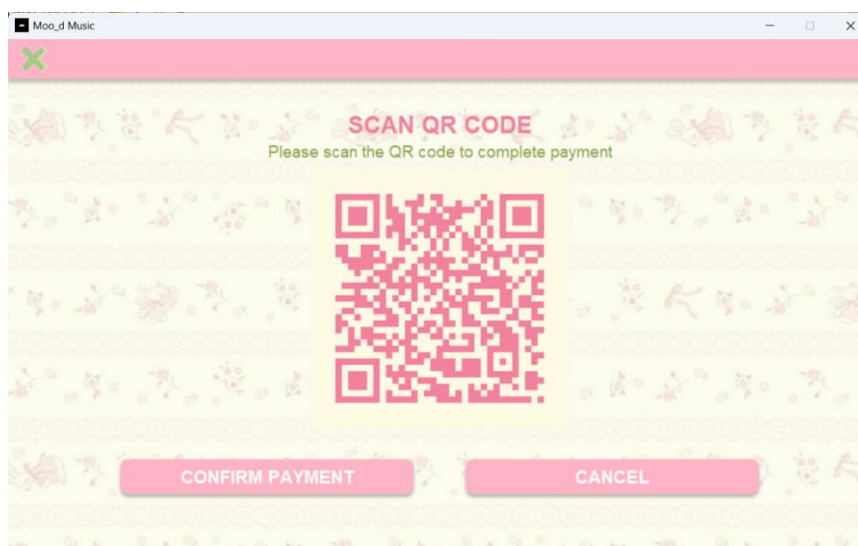
Giao diện tiếp theo là giao diện thanh toán, được gọi lên khi người dùng ấn chọn một bài hát chưa sở hữu bất kì, giao diện sẽ có ba phần chính, bao gồm khung hiển thị thông tin bài hát đang tiến hành mua, thông tin của người dùng và ô lựa chọn phương thức thanh toán.



Hình 26: Giao diện khi thanh toán bài hát

Người dùng có thể ấn vào dấu X để đóng giao diện này lại và hủy tiến trình mua, còn nếu người dùng bấm nút “Purchase Now”, một màn hình bao gồm mã thanh toán sẽ được gọi lên. Mã này không thực sự thanh toán được mà chỉ tượng trưng để quét

thông tin giao dịch, thay vào đó người dùng sẽ dùng nút “Comfirm Payment” để hoàn thành tiến trình mua, app sẽ hiển thị thông báo thanh toán thành công và màn hình sẽ được tự động hủy sau 2 giây để trở về với màn hình trước đó, các thông tin giao dịch cũng sẽ được thêm mới trong cơ sở dữ liệu.

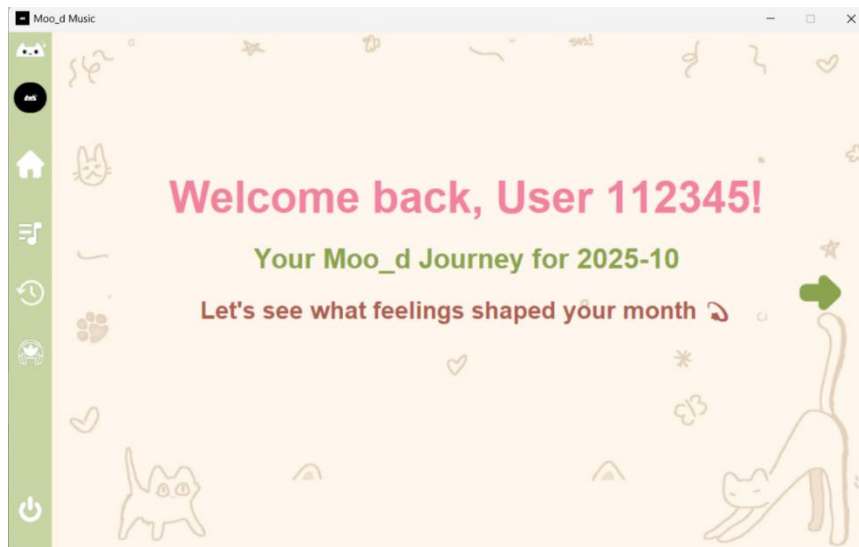


Hình 27: Giao diện xác nhận thanh toán

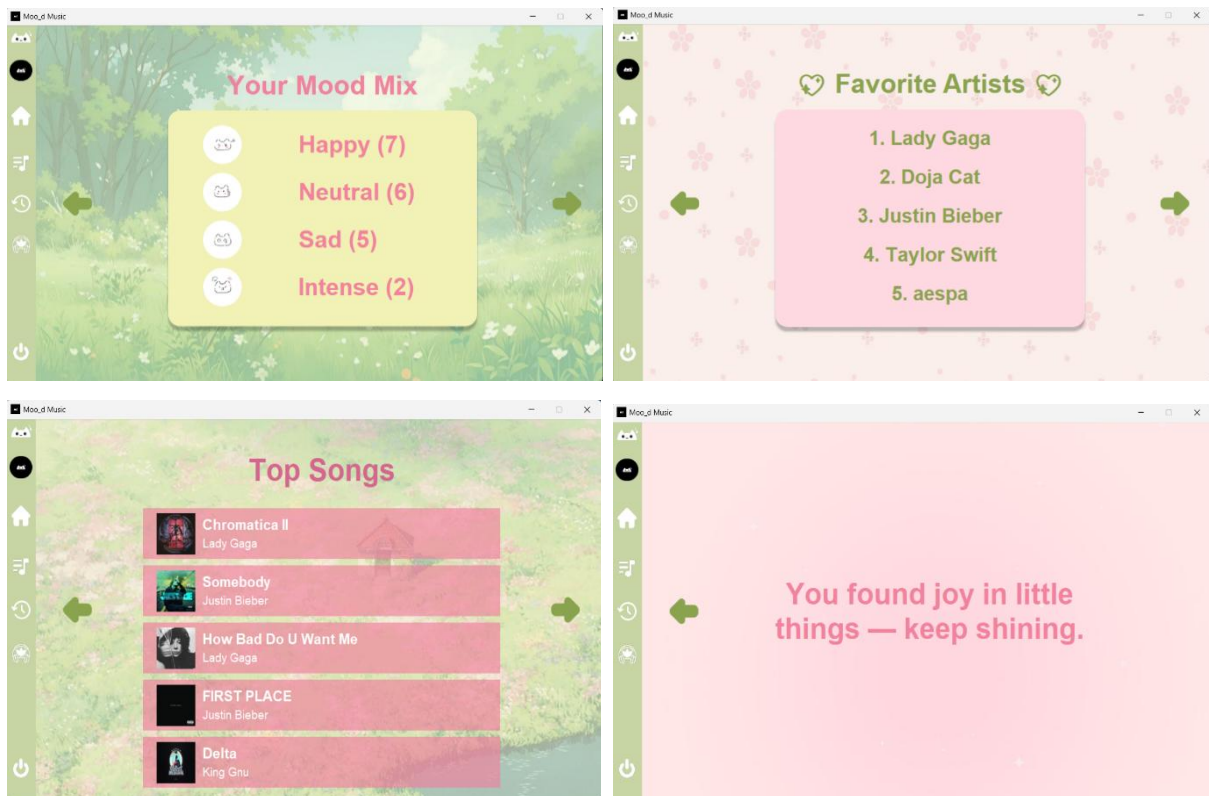
4.1.9 Giao diện Wrap Up

Điều đặc biệt ở app Moo_d là được thêm chức năng Wrapped Up, đây là chức năng đặc biệt được tạo ra với mong muốn giúp người dùng kết nối hơn với bản thân khi có thể mở lại “món quà” ghi dấu lại hành trình cảm xúc của tháng vừa qua. Qua đó người dùng sẽ có sự gắn kết với ứng dụng và cảm nhận rõ ràng hơn sự đồng điệu giữa tâm trạng và âm nhạc. Đó cũng chính là mục đích lớn nhất khi nhóm em quyết định phát triển lên ứng dụng nghe nhạc Moo_d.

Giao diện Wrap Up bao gồm 5 trang nhỏ. Trang đầu tiên sẽ là lời chào thân thương và lời mời gọi để người dùng bắt đầu nhìn lại hành trình cảm xúc của mình của tháng vừa rồi. Trang thứ hai sẽ hiển thị những mood chủ đạo mà người đã “tâm sự” cùng với ứng dụng. Và cũng không thể thiếu giao diện bảng xếp hạng 5 nghệ sĩ yêu thích nhất và 5 bài “tủ” mà người dùng đã nghe đi nghe lại. Và cuối cùng sẽ là một câu quote bất kì dựa vào cảm xúc chủ đạo gửi tới người dùng như một lời động viên và kết thúc hành trình.



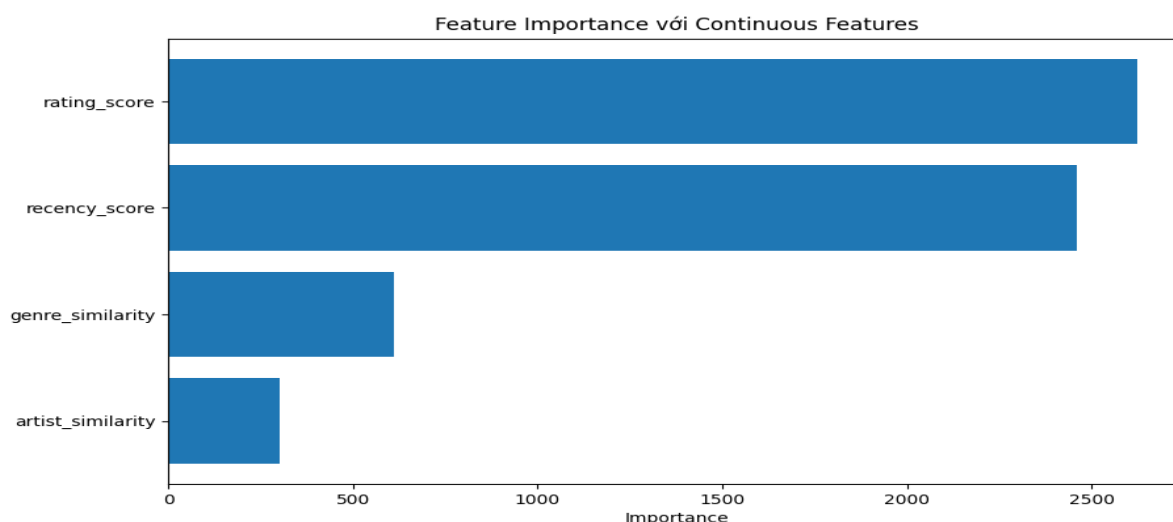
Hình 28: Giao diện Wrap Up



Hình 29: Các trang tiếp theo trong giao diện Wrap Up

4.2. Đánh giá mô hình

4.2.1 Mô hình Recommend for today



Hình 30: Biểu đồ phân chia mức độ đóng góp của các yếu tố

Biểu đồ trên phân chia mức độ đóng góp của các yếu tố trong mô hình học máy. Nhìn vào hình thì có thể thấy điểm đánh giá và thời gian phát hành có ảnh hưởng tới danh sách gợi ý lớn hơn so với điểm tương đồng nghệ sĩ, thể loại. Nhưng như vậy mới hợp lý vì mục tiêu của mô hình là “cf_score” - tức mức độ yêu thích chung trên toàn hệ thống. Vì đã có các mục gợi ý riêng liên quan đến thể loại, nghệ sĩ tương đồng là Recommend genres, Recommend artist nên phần Recommend for today muốn giới thiệu cho người dùng các bài hát phổ biến trên nền tảng nhiều hơn là tập trung cá nhân hóa.

Đánh giá mô hình LGBM:
R² score: 0.8581
RMSE: 0.1054
MAE: 0.0703

Hình 31: Kết quả đánh giá mô hình LGBM

Chỉ số R-squared = 0,8581 cho thấy mô hình giải thích được 85,81% lý do bài này phổ biến hơn bài kia thông qua 4 feature, chỉ có số ít còn lại là do yếu tố khác không giải thích được.

RMSE đo độ sai khác trung bình giữa giá trị dự đoán và giá trị thực. Với mô hình này, “cf_score” nằm trong khoảng từ 0 đến 1, mà RMSE = 0,1054 tức là giá trị dự đoán

chỉ chênh lệch trên dưới 0,1054 so với giá trị thực. Đây là con số nhỏ cho thấy mô hình đang dự đoán khá phù hợp.

MAE đo mức độ sai số trung bình theo giá trị tuyệt đối cũng chỉ ra mô hình dự đoán chỉ lệch 7% so với giá trị thực. Tổng kết lại, mô hình đang đi đúng hướng với mục tiêu đề ra ban đầu.

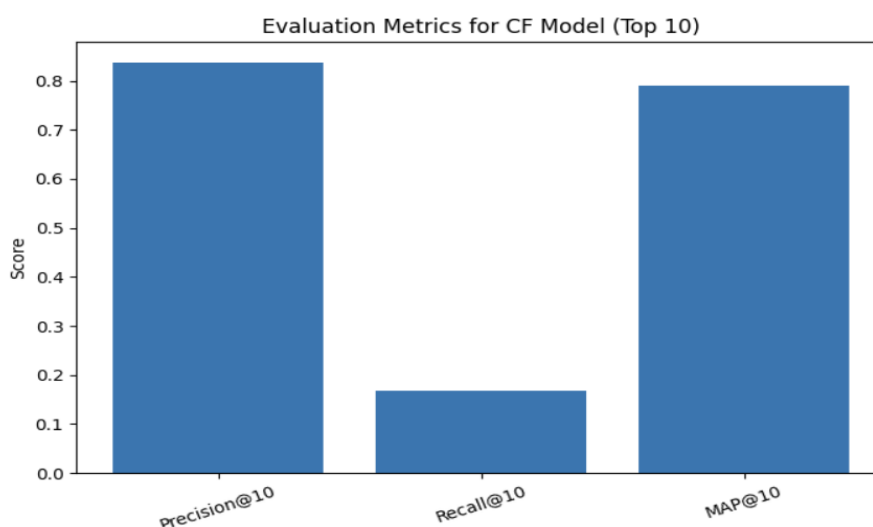
4.2.2 Mô hình Recommend artist

```
K = 10
precision_scores = []
recall_scores = []
map_scores = []
test_users = list(ground_truth.keys())
num_eval_users = min(100, len(test_users))
```

Ở bước đánh giá mô hình này, để đảm bảo được sự khách quan, ta chọn ngẫu nhiên 100 user để đánh giá trên tổng số 1000 người `num_eval_users = min (100, len(test_users))`, đảm bảo 100 người này không trùng lặp với nhau. `rec_df = recommend_for_user_EVAL(...)` sẽ trả về kết quả là 10 nghệ sĩ theo mô hình dự đoán. Đối với những người dùng mới thì mô hình sẽ chỉ gợi ý sang những nghệ sĩ phổ biến nhất trong hệ thống. Đối với mô hình này, ta sẽ đánh giá dựa vào những chỉ số: `Precision@K`, `Recall@K`, và `MAP` với `K = 10`

Từ các chỉ số trên, nhóm dự án thấy được việc kiểm định mô hình gợi ý KNN Collaborative Filtering (KNN CF) trên tập dữ liệu kiểm thử cho kết quả khá tốt về độ chính xác và khả năng xếp hạng của những gợi ý. Cụ thể, `Precision@10` đạt trung bình 0.8370, nghĩa là hơn 83% trong số 10 nghệ sĩ mà mô hình đề xuất cho mỗi người dùng là vô cùng phù hợp và bám sát với sở thích nghe nhạc của họ. `MAP@10` cũng đạt một con số ấn tượng là 0.7894, cho thấy mô hình không chỉ gợi ý đúng “gu” và còn ưu tiên xếp những nghệ sĩ phù hợp lên các vị trí đầu tiên, giúp mang lại trải nghiệm người dùng tốt hơn. Tuy nhiên chỉ số `Recall@10` chỉ đạt 0.1687, nghĩa là tổng số nghệ sĩ mà người dùng đã tương tác thì mô hình chỉ ra được khoảng 16%. Điều này cũng không phải vấn đề quá nghiêm trọng, vì mục tiêu chính của `Moo_d` vẫn là tuy đưa ra ít đề xuất nhưng

mà mỗi đề xuất phải thực sự chất lượng, và phù hợp với người dùng nhất có thể thay vì cố gắng đề xuất hết toàn bộ nghệ sĩ.



Hình 32: Biểu đồ đánh giá mô hình CF (Top 10)

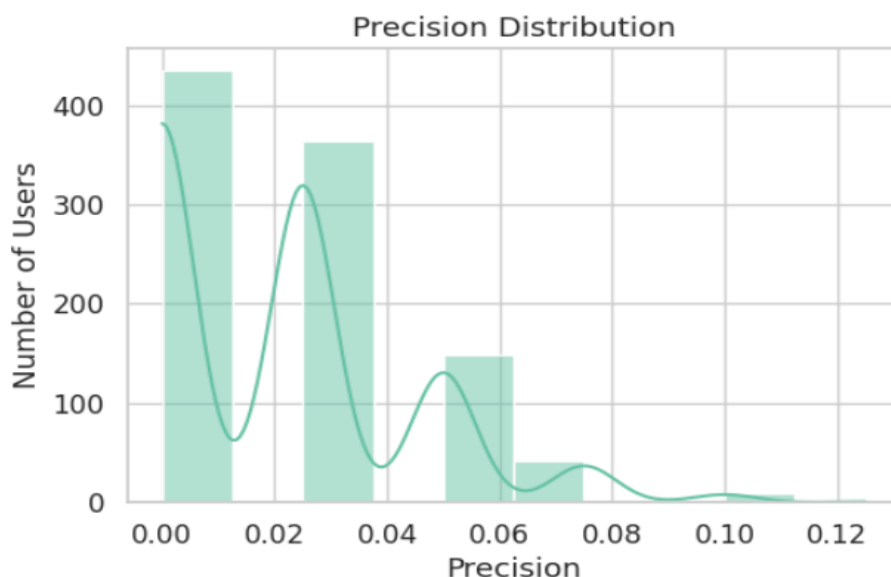
Tổng kết lại, mô hình gợi ý nghệ sĩ đang hoạt động vô cùng hiệu quả với khả năng đưa những đề xuất khá tốt, ổn định với độ chính xác cao và thứ tự gợi ý cũng vô cùng hợp lý. Chỉ số Precision và MAP vượt trội cho thấy mô hình đang được ứng dụng tốt vào Moo_d. Trong tương lai, vẫn có thể tiếp tục cải thiện thêm chất lượng mô hình nếu tập dữ liệu bổ sung thêm nhiều yếu tố như mood hiện tại của người dùng hay triển khai thành mô hình Hybrid (CF + Content Based) với những thuộc tính như quốc gia, tempo,...

4.2.3 Mô hình Recommend genres

Đối với mô hình thứ ba của đề tài - mô hình gợi ý các bài chưa mua theo thể loại - nhóm lựa chọn đánh giá bằng chỉ số Precision@K.

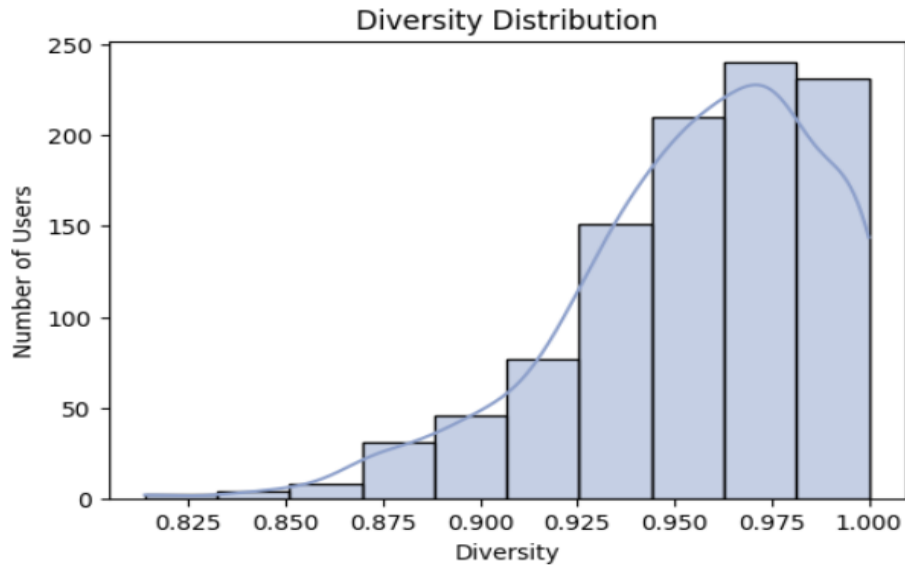
```
hits = set(all_recommended) & set(ground_truth)
precision = len(hits) / len(all_recommended) if all_recommended else 0
hit_rate = 1.0 if len(hits) > 0 else 0.0
diversity = calculate_intra_list_diversity(all_recommended, master_features)
```

Chỉ số Precision@K sẽ tính tỷ lệ trong số lượng bài mà mô hình gợi ý, tối đa là 40 bài trên một người dùng, thì có bao nhiêu bài trùng với các bài thuộc tập dữ liệu kiểm thử của người dùng đó, nghĩa là ta sẽ biết được người dùng thực sự mua và tương tác với các gợi ý mà mô hình đưa ra hay không. Kết quả đo lường trung bình của chỉ số Precision@K được chạy trên toàn bộ 1000 người dùng xấp xỉ 0.021, tức khoảng 2%.



Hình 33: Biểu đồ phân phối Precision của người dùng

Nhìn vào biểu đồ histogram này, có thể thấy trên 1000 người dùng thì có đến hơn 2/3 người dùng không có tương tác với các bài được gợi ý và nếu có thì đa phần rơi vào khoảng 1 đến 2 bài trên tổng cộng 40 bài. Tuy nhiên nếu xét theo tiêu chí có bao nhiêu người dùng tương tác với ít nhất 1 bài thì tỷ lệ đạt 56.5%, một con số khá phù hợp cho một mô hình gợi ý nhạc. Ngược lại với độ chính xác, độ đa dạng của các gợi ý thì mô hình lại đáp ứng rất tốt, chỉ số đo lường độ đa dạng giữa các bài được gợi ý với nhau đạt 95.5%, được xác định bằng $(1 - \text{mean}(\text{cosine_similarity}))$.



Hình 34: Biểu đồ phân phối độ đa dạng giữa các bài được gợi ý

Tổng kết lại, chỉ số Precision thấp cho biết mô hình này chưa mang lại hiệu quả cao; mặc dù các gợi ý đưa ra có thể rất đa dạng và khác lạ, nhưng lại không phải là những bài mà người dùng đang thực sự tìm kiếm. Nhóm nhận ra vấn đề này phần lớn xuất phát từ hai hạn chế cốt lõi về dữ liệu người dùng và đặc trưng đầu vào. Thứ nhất, về dữ liệu người dùng, việc sử dụng bộ dữ liệu giả lập được tạo ngẫu nhiên khiến các tương tác giữa người dùng và bài hát không hoàn toàn phản ánh thói quen nghe nhạc thực tế. Điều này làm ảnh hưởng mạnh đến độ chính xác của mô hình do đang phải dự đoán những hành vi không theo quy luật nào. Bên cạnh đó, dữ liệu về các bài hát mà API cung cấp hiện tại chỉ gồm các thông tin cơ bản như thể loại và tên nghệ sĩ, mà chưa khai thác được những yếu tố âm nhạc quan trọng khác như nhịp độ hay giai điệu bài hát. Việc thiếu các đặc trưng quan trọng này sẽ giới hạn khả năng đề xuất những bài hát thực sự đồng điệu với gu nghe nhạc và sở thích cá nhân của người dùng. Nhóm tin rằng nếu vấn đề về dữ liệu được cải thiện thì mô hình sẽ chứng minh được tính chính xác của mình một cách công bằng hơn.

4.2.4. Mô Hình Recommend player

Nhóm quyết định thực hiện kiểm định chất lượng cho mô hình thuần content-based filtering bằng chỉ số NDCG. Đây là chỉ số tính được từ việc so sánh Recommendations của mô hình với Ground Truth là mục đích thực sự của mô hình ở

đây chính là các bài mà user sẽ mua, vì tất cả dữ liệu là giả định và không có dữ liệu thật nên đã tạo một ground truth giả định để đánh giá bằng cách sử dụng content similarity làm relevant, so sánh Recommendations với bài đang được phát, nếu như cùng Artist sẽ cộng 2 điểm, cùng genre và mood phù hợp sẽ cộng 1 điểm sau đó tính DCG và từ đó sẽ tính điểm theo NDCG theo thước đo này, giả định này sẽ không đúng hoàn toàn với user-behavior nhưng vẫn có thể sẽ phù hợp được dùng để đánh giá mô hình content-based này.

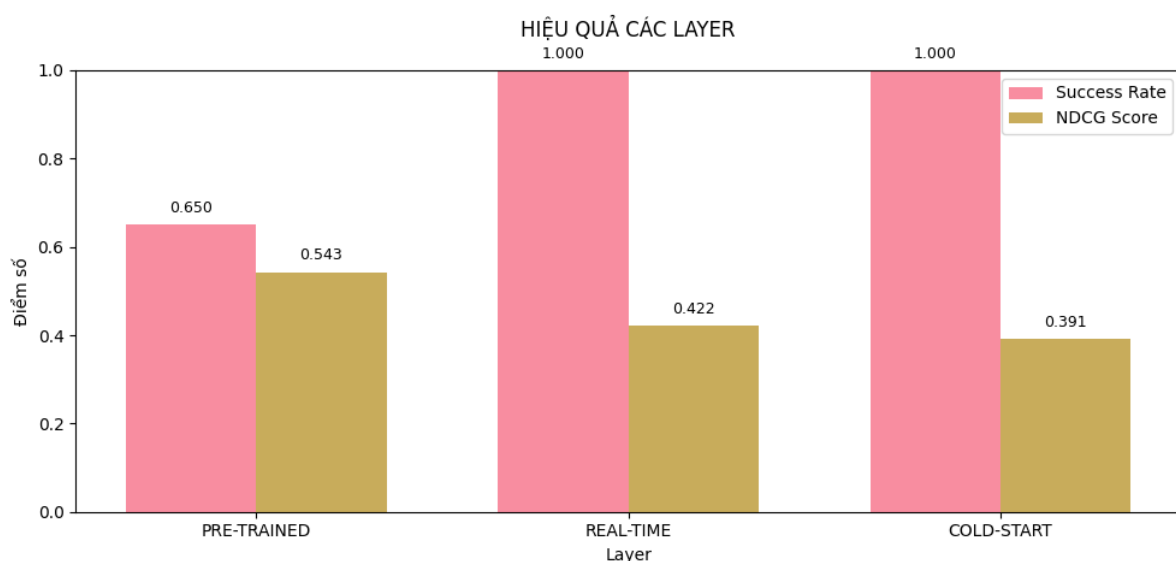
Bước đầu tiên tạo ra các trường hợp user để test dữ liệu cho 3 lớp, lớp 1 sẽ là các bài hát có trong database và nằm trong similarity dict, lớp 2 là các bài tương tự nhưng sẽ không nằm trong top 100 bài nổi tiếng, lớp 3 là các user mới và các bài hát không có trong similarity dict.

Created 40 DIVERSE scenarios:

- Layer 1 (Pre-trained): 13 scenarios
- Layer 2 (Real-time): 13 scenarios
- Layer 3 (Cold-start): 14 scenarios

Hình 35: 3 lớp mô hình Recommend player

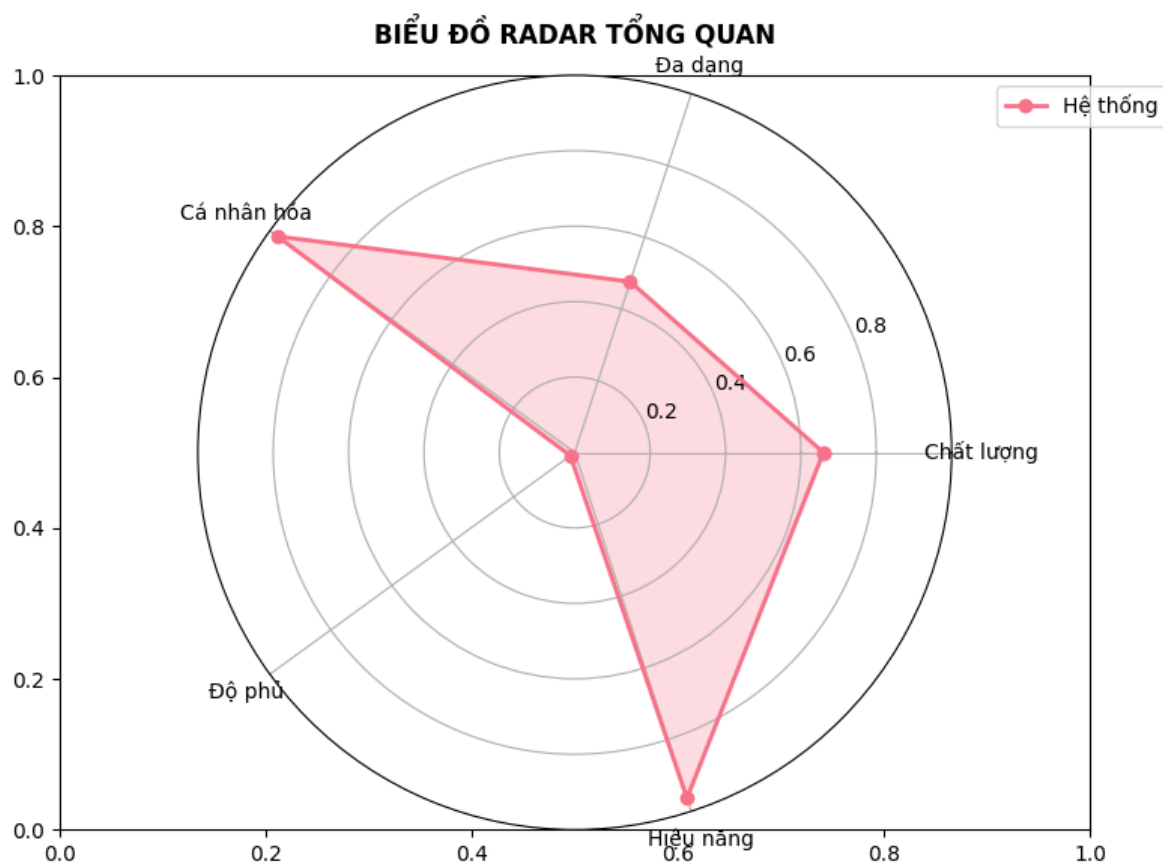
Sau khi chạy test riêng cho từng lớp mô hình ta có kết quả như sau:



Hình 36: Kết quả chất lượng mô hình theo từng lớp

Kết quả này đang tính riêng cho từng track, lớp Pre-trained đang có kết quả tốt nhất với 54.3% thứ tự đề xuất các bài hát đúng nhất với thứ tự user muốn mua các bài đó, tuy nhiên tỷ lệ thành công ở lớp này chỉ là 65% vì dính phải các bài nằm ngoài similarity dict sẽ bị fall back qua lớp 2, ở lớp 2 chỉ số NDCG thấp hơn vì đây là class fallback sau khi bài được chọn là một bài không nổi nằm ngoài ma trận, nên các thuật toán có phần đơn giản hơn lớp pre-trained tuy nhiên đây là lớp quan trọng đảm bảo cho user vẫn sẽ được đề xuất nhạc ở mọi trường hợp tỷ lệ thành công luôn là 100%, lớp cuối cùng cold start dành cho người dùng mới, các dữ liệu có được vẫn chưa nhiều, chủ yếu các thuật toán đề xuất các bài nổi tiếng và ép về đa dạng các thể loại nhạc nên NDCG không cao là điều có thể hiểu được tuy nhiên đây cũng là lớp fallback là cuối cùng cho 2 lớp trên vẫn sẽ đảm bảo user sẽ có được đề xuất nhạc với tỷ lệ thứ hạng lý tưởng với họ cũng đã đúng gần 3 bài trên 8 bài được gợi ý ra.

Sau khi chạy test riêng cho từng lớp, ta chạy test cho tổng mô hình cả 3 lớp



Hình 37: Kết quả chất lượng mô hình

Có thể thấy NDCG@8 lần này có cải thiện hơn so với việc test riêng từng lớp, vì 3 lớp này nếu hoạt động độc lập có thể không mang lại kết quả cao nhưng khi được gộp vào chung 1 mô hình chúng hỗ trợ các trường hợp cho nhau rất tốt, kết quả ra rằng trong 8 bài được đề xuất trung bình có đến gần 5 bài được xếp đúng với thứ hạng mà user mong muốn, đây là kết quả hiểu được với việc thiết lập ground truth như trên đối với một mô hình thuần là content-based.

Chỉ số “Intra-list Diversity” cũng ở mức khá cho thấy rằng trong số 8 bài được đề xuất có đến xấp xỉ 4 nghệ sĩ khác nhau, mô hình content-based vẫn cân bằng giữa việc đề xuất nội dung gần với bài đang phát bên cạnh đó vẫn có tính đa dạng bên trong để không làm user bị nhàm chán khi cứ có 1 nghệ sĩ xuất hiện hoài.

Chỉ số “Personalization” cao và vượt ngoài mong đợi với mục đích của mô hình là mong muốn cá nhân hóa đề xuất bài nhạc cho người dùng với từng bài hát mà người đó chọn, đây là con số ấn tượng khi có đến 97.6% đề xuất cho các người dùng test là riêng biệt và không trùng lặp nhau.

Cuối cùng chỉ số Coverage thấp là điều có thể lý giải được, vì hiện tại trong database đang có đến gần 17900 bài hát riêng biệt, chỉ số này cho thấy với mỗi lần đề xuất 8 bài ta đề xuất 40 lần nghĩa là 320 bài trong số đó có 268 bài là khác nhau, hoàn toàn ổn và hợp lý với mô hình này.

Kết luận cuối cùng, mô hình hiện tại có các chỉ số là ổn so với kỳ vọng và mục đích nhóm mong muốn đối với mô hình ở giao diện đĩa xoay này. Tiêu chí hàng đầu chính là cá nhân hóa cho từng user đã đạt được với 97.6% và mức độ phù hợp với user đó vẫn là ổn với NDCG@8 62.4%.

CHƯƠNG 5: KẾT LUẬN VÀ ĐỀ XUẤT GIẢI PHÁP

5.1 Điểm mạnh của dự án

Mặc dù còn nhiều giới hạn về dữ liệu và hạ tầng, dự án vẫn đạt được những kết quả tích cực với một số điểm mạnh đáng ghi nhận. Một trong những điểm nổi bật của dự án nằm ở phần giao diện người dùng. Ứng dụng được thiết kế với tông màu nhẹ nhàng gồm đầy đủ các chức năng đáp ứng cho việc nghe nhạc. Nhưng không chỉ dừng lại ở những tính năng cơ bản, ứng dụng còn sở hữu một số đặc trưng riêng như cá nhân hóa đề xuất của ứng dụng theo tâm trạng, có thể tạo và quản lý danh sách phát cũng như theo dõi thói quen nghe nhạc của bản thân thông qua chức năng báo cáo hàng tháng.

Song song với đó, dự án cũng đã được tích hợp thành công các mô hình gợi ý vào ứng dụng và cho ra kết quả ở mức khá tốt. Dù mô hình còn nhiều điểm cần tối ưu, việc áp dụng được các thuật toán học máy vào ứng dụng là một kết quả tích cực và là nền tảng quan trọng để nhóm tiếp tục cải thiện trong các phiên bản kế tiếp.

5.2 Điểm yếu của dự án

Dự án đã được thực hiện thành công với những kết quả khả quan nhưng nhìn chung vẫn tồn tại nhiều điểm yếu cần cải thiện tốt hơn. Đầu tiên, điểm yếu lớn nhất của nhóm chính là nằm ở tệp dữ liệu hiện tại. Do nhóm đã giả lập dữ liệu nên nó chưa phản ánh chính xác được hành vi và thói quen nghe nhạc thực tế của người dùng. Điều này đã tác động trực tiếp đến khả năng triển khai và mức độ hoàn thiện của mô hình học máy. Điểm yếu thứ hai, là do sử dụng API từ Itunes nên bộ dữ liệu được cấp thiếu nhiều thuộc tính quan trọng như chi tiết bài hát, đặc điểm của nghệ sĩ, thể loại, quốc gia, nhịp điệu,...khiến cho nhóm chưa tìm ra hết tiềm năng của mô hình học máy. Ngoài ra mỗi thành viên trong nhóm đều đang chạy dự án trên localhost, chưa đồng bộ dự án lên cloud, nên nhóm đã gặp khó khăn trong việc đồng bộ dữ liệu và chạy mô hình theo thời gian thực. Tốc độ tải mô hình và hiển thị lên ứng dụng vẫn còn gặp tình trạng giật lag, khởi tạo chậm do giới hạn phần cứng, và khối lượng dữ liệu quá nhiều. Nhóm sẽ tiếp tục cố gắng cải thiện tệp dữ liệu cũng như tối ưu thuật toán, hạ tầng triển khai để dự án có kết quả tốt hơn.

5.3 Hướng phát triển trong tương lai

Trong dự án, mục tiêu kết hợp các mô hình học máy vào ứng dụng Moo_d của nhóm đã thành công, tuy nhiên ứng dụng vẫn có các khuyết điểm, vì vậy nhóm đã vạch ra một kế hoạch về hướng phát triển của Moo_d.

Đầu tiên, nhóm cần khắc phục vấn đề về metadata của bài hát, đây là yếu tố cốt lõi quyết định đến độ hiệu quả của các mô hình đề xuất khi thiếu đi các dữ liệu đặc trưng riêng biệt của một bài hát. Nhóm đề xuất sử dụng API của bên thứ 3 khác với đầy đủ metadata hơn là Spotify for Developers cung cấp các đặc trưng âm thanh chi tiết hơn về một bài nhạc.

Tiếp đến nhóm muốn đi sâu hơn vào việc tối ưu các tính năng về Mood của Moo_d. Đầu tiên cải tiến thêm việc sẽ không chỉ có 4 mood cơ bản nữa mà sẽ thực sự để người dùng tự do chọn lựa mood theo sở thích của mình một cách năng động hơn. Bên cạnh đó thêm vào một mô hình máy học về mood, mô hình này thông qua học các đặc tính ẩn bài hát từ API của Spotify sau đó tự đề xuất mood cho người dùng khi gán mood cho bài hát người dùng có thể tự gán mood theo sở thích của mình mà vừa có thể nhận được gợi ý từ Moo_d. Thêm vào đó nữa với việc sử dụng các thông tin từ mood tracker, như là thời gian chọn mood đăng nhập từ đó phát triển thêm tính năng đề xuất mood của người dùng theo ngữ cảnh dựa vào thói quen sử dụng của người dùng. Mô hình có thể học được như rằng người này sẽ hay nghe nhạc vui vào buổi sáng, nhạc thư giãn vào buổi tối, nhạc sôi động khi đang tập thể dục từ đó sẽ tự động đề xuất mood tracker cho người dùng khi đăng nhập vào ứng dụng.

Cuối cùng sẽ chuyển việc quản lý dữ liệu hiện tại từ local lên cloud để đồng bộ hóa dữ liệu của ứng dụng, cũng như gán index cho các khóa chính hay dùng để tối ưu hóa tốc độ query lên khắc phục việc ứng dụng hiện tại mất nhiều thời gian để truy vấn thực hiện các chức năng hiển thị lên giao diện. Thêm vào đó nữa cải tiến thêm các tính năng tự động ví dụ như qua một khoảng thời gian (1 tháng, 2 tháng,...) sẽ tự động pre-train một số mô hình lại nhằm đảm bảo chất lượng bài hát được đề xuất ra, cũng như tự động tính toán và cập nhật các bảng xếp hạng bài hát thịnh hành theo thời gian, có thể là theo tuần hoặc là theo tháng.

5.4 Kết luận

Thông qua việc thực thi các mô hình gợi ý và tích hợp vào Moo_d, nhóm đã tự tìm hiểu được nhiều kiến thức mới liên quan đến Machine Learning. Tuy còn nhiều thiếu sót, nhưng phiên bản ứng dụng hiện tại đã có cải tiến rất lớn so với phiên bản lần trước. Đây sẽ là cơ sở vững mạnh của nhóm trong trường hợp mở rộng, tối ưu và cải tiến ứng dụng hơn nữa sau này.

PHỤ LỤC

Đường dẫn bộ dữ liệu, slide, bảng phân công và video demo dự án:

<https://drive.google.com/drive/folders/1w0-L5WDPU8mU8iu6VNmHQ5qqjahtBRSR>

Đường dẫn code của dự án:

https://github.com/vulephuongthuy/MachineLearning_Nhom7.git

TÀI LIỆU THAM KHẢO

- Aggarwal, C. C. (2016). *Recommender systems: The textbook* (Vol. 39). Springer.
- Bhutada, R. M., Kadam, P., Bhople, S., Jangid, R., Gavatre, V., & Deokar, A. (2023). Survey on Emotion-Based Song Recommendation System. *nternational Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, 12(4), 3677. doi:10.15680/IJIRSET.2023.1204199
- Biswal, A., Borah, M. D., & Hussain, Z. (2021). Music recommender system using restricted Boltzmann machine with implicit feedback. In M. Zelkowitz, *Advances in computers* (Vol. 121, pp. 367–402). Academic Press.
- Hướng dẫn đầy đủ về cách sử dụng LightGBM dành cho người mới. (2025, February 24). Retrieved November 2025, from FUNiX: <https://funix.edu.vn/chia-se-kien-thuc/huong-dan-day-du-ve-cach-su-dung-lightgbm/>
- Leskovec, J., Rajaraman, A., & Ullman, J. D. (2020). Recommendation systems. In J. R. Leskovec, *Mining of massive datasets* (3rd ed. ed.). Cambridge University Press.
- LightGBM (Light Gradient Boosting Machine). (2025, July 15). Retrieved November 2025, from GeeksforGeeks: <https://www.geeksforgeeks.org/machine-learning/lightgbm-light-gradient-boosting-machine/>
- Lu, J., & Wu, M. (2025). Design and application of a music recommendation system based on user behavior and feature recognition. *Systems and Soft Computing*, 7, 200274. doi:10.1016/j.sasc.2025.200274
- Madhan, M., Livin Kumar, K., Mohammed Abdul Saheen, P., & Jenifer, P. (2025, May). Emotion-Aware Music Recommendation System Using Deep Learning. *INTERNATIONAL JOURNAL OF NOVEL RESEARCH AND DEVELOPMENT*, 10(5), c805–c810. Retrieved from <https://ijnrd.org/papers/IJNRD2505273.pdf>

- Ricci, F., Rokach, L., Shapira, B., & Kantor, P. B. (2011). Recommender systems: An overview. *AI Magazine*, 32(3), 13-18.
- Wang, S., Xu, C., Ding, A. S., & Tang, Z. (2021). A Novel Emotion-Aware Hybrid Music Recommendation Method Using Deep Neural Network. *Electronics*, 10(15), 1769. doi:10.3390/electronics10151769